Rings: An Algebraic Structured Peer-to-Peer Network for Decentralized Identity and Resource Ownership

Ryan J. Kung

Abstract

Rings is an implemented algebraic structured peer-to-peer network for decentralized identity, resource ownership, transport, storage, privacy, and application protocols. The project starts from a concrete failure mode in contemporary Internet systems: application servers, DNS names, relay services, and platform accounts often become the authority that identities and data must trust. Rings instead treats identities and resources as algebraic objects over a structured CorrectChord overlay, with browser and native runtimes connected through WebRTC/WebAssembly adapters and cryptographic protocols kept on carriers separate from the routing identifier space. This consolidated Rings 1.0 whitepaper unifies the foundational algebraic model with six production subsystem specifications: range-proved finger convergence, non-transitive WebRTC Chord routing and bridge-delegated delivery, CRDT removal stability and per-writer data provenance, durable local records and age-gated replicated relay custody, cleanest-first slot accounting and node/edge/lane load bounds, and the Sphinx/AEZ-v5 onion Kleisli pipeline, together with the 1.0 service receipt bridge to the companion DRanking 2.0 protocol. The core object is

$$\mathcal{R}\rangle\backslash\}\int = (R, \mathcal{R}_N, \mathbf{Set}/R, \Omega_N, \mathsf{T}_A^{\mathcal{E}}, \Pi, \mathbb{G}, \mathcal{X})$$

with

$$\begin{aligned} R &= \mathbb{Z}/2^{160}\mathbb{Z}, \\ \mathcal{R}_N &= \text{Correct-Chord overlay object}, \\ \Omega_N(X,H) &= (X, \text{owner}_N \circ H), \\ \tau_\nu &: S_\nu \times I_\nu \to \mathsf{T}_{A_\nu}^{\mathcal{E}_\nu}(S_\nu \times O_\nu). \end{aligned}$$

$$\{D, V, M, H, Q, W\} \subset \text{Obj}(\mathbf{Set}/R), \qquad \mathsf{CryptoCarrier} \cap R = \varnothing.$$

I. Introduction

Centralized application platforms simplify deployment, but they also concentrate identity, naming, storage, and traffic control at service operators. Cloud and platform centralization therefore becomes a privacy, availability, and autonomy risk for users who need direct communication and durable control over their data [1]–[3]. The Rings project is motivated by this observation: a sovereign network should let a participant own an identity, establish a session, route to another identity or resource, and run application protocols without turning a single application server into the root of authority.

Rings is implemented as a network, but this paper is organized around the algebraic structure that makes the implementation coherent. It states the ideal carrier, ownership, transition, durability, admission, and privacy laws that the Rings 1.0 implementation realizes. The older intuition was that Decentralized Identifiers (DIDs) directly form a finite ring suitable for all routing and cryptographic operations. The refined model is more precise: Chord identifiers form an additive circular carrier for placement, interval order, replica selection, and finger targets; threshold schemes, signatures, encryption, wide-block onion encipherment, and zero-knowledge protocols use their own fields or groups.

The resulting design has three practical consequences. First, DIDs and virtual DIDs (VIDs) share the same 160-bit placement surface, so identities, mailboxes, service descriptors, chunks, and application rendezvous points can be routed by one owner function. Second, Rings builds on Chord and CorrectChord rather than replacing the overlay literature: successor lists, multi-predecessor relations, join, rectify, stabilize, range-proved finger convergence, and passive 1-hop gap-bridge delegation are the maintenance substrate on which Rings semantics are layered [4]–[7]. Third, state transitions return typed effects instead of directly performing network IO, which allows the overlay, storage, durability, load-accounting, and onion-pipeline state machines to be reasoned about purely, model-checked with Stateright [8], and then interpreted by browser or native runtimes.

II. Problem Statement and Design Goals

Rings addresses a narrow but recurring gap between existing decentralized systems. Kademlia-derived systems

Ryan J. Kung can be reached at ryankung@ieee.org. Manuscript created February 8, 2023; consolidated for Rings 1.0 November 2026.

provide scalable lookup, but the XOR metric does not by itself give Rings the circular ownership law needed for successor lists, replicated ownership, and interval-based placement. Relay networks and mixnets improve metadata resistance, but they are not structured P2P ownership systems. Blockchains provide global ordering, but embedding consensus in every message or storage operation would turn Rings into a ledger rather than an algebraic structured peer-to-peer substrate.

The problem is therefore to specify a network object

$$\mathcal{R}\rangle\backslash\}\int$$

that simultaneously supports:

1) self-certifying identity and delegated sessions;
2) structured lookup, range-proved finger convergence, and non-transitive WebRTC routing;
3) browser-native and native transport and crash-durable storage surfaces;
4) CRDT storage with tombstone stability, per-writer cryptographic provenance, and replicated relay custody;
5) Sybil-resistant slot accounting, credit reclaim, and bounded load under free identities;
6) Sphinx/AEZ-v5 onion Kleisli pipelines and counterparty-signed service receipts.

This paper makes the following contributions. It presents Rings as an algebraic structured peer-to-peer network, modeling the identifier space as the additive cyclic group $R = \mathbb{Z}/2^{160}\mathbb{Z}$ and lifting resource placement into $\mathbf{Set}/R$. It specifies range-proved finger convergence and bounded multi-predecessor non-transitive Chord routing over restricted WebRTC graphs (Section IV). It establishes the removal-stability law, HLC data provenance, factored carrier eviction, and signed lease heartbeats for replicated CRDT storage (Section V). It defines the crash-meaning authority lattice $C \times M$, presence markers, per-platform crash durability, group commit, link-local replay windows, and age-gated replicated relay custody (Section VI). It proves fair slot accounting across five shared node tables, credit reclaim, and single-charge chunk reassembly under free identities (Section VII). It replaces legacy relay framing with client-sealed Sphinx/AEZ-v5 onion Kleisli pipelines (Section VIII), bridges 1.0 service receipts into the companion DRanking 2.0 protocol [9] (Section IX), and summarizes the Stateright model-checking and analytical complexity bounds across all Rings 1.0 subsystems (Section X).

III. Algebraic System Model, Interfaces, and Effects

A. Carrier and Overlay Model

Assumption 1 (System scope).

$$\begin{aligned}
\mathsf{fault} &= \text{fail-stop},\\
\mathsf{auth}(m,\sigma,k) &= V(k,\sigma,m),\\
\mathsf{net} &= \text{async},\\
\mathsf{io} &= I_X : A_X^* \to \mathsf{IO},\\
\mathsf{consensus} &\notin \mathsf{Overlay},\\
\mathsf{order} &= \mathsf{SidecarWitness},\\
\mathsf{crypto_ops} &\cap \mathsf{route_ops} = \varnothing.
\end{aligned}$$

The fail-stop clause scopes overlay maintenance; identity, storage provenance, load accounting, onion routing, ranking, and application protocols carry explicit adversarial predicates at their own layers.

Definition 1 (Identifier carrier).

$$\begin{aligned}
M &= 2^{160},\\
R &= \mathbb{Z}/M\mathbb{Z},\\
\delta_a(x) &= (x-a) \bmod M,\\
(a,b]_R &= \{x \in R : 0 < \delta_a(x) \le \delta_a(b)\},\\
\mathsf{RouteOps}_R &= \{0,+,-,\delta_a,(_,_]_R\},\\
\mathsf{Mult}_R &\notin \mathsf{RouteOps}_R.
\end{aligned}$$

Definition 2 (Live topology).

$$\begin{aligned}
N &\subset R, \quad N \neq \varnothing,\\
\mathrm{owner}_N(k) &= \arg\min_{u \in N} \delta_k(u),\\
\mathrm{pred}_N(n) &= \arg\min_{p \in N\setminus\{n\}} \delta_p(n),\\
\mathrm{Succ}_N^q(n) &= \mathrm{take}_q\big(\mathrm{sort}_{\delta_n}(N \setminus \{n\})\big),\\
\delta_a|_N &= \mathsf{injective}.
\end{aligned}$$

Assumption 2 (Correct-Chord stable base).

$$\begin{aligned}
r &= |\,\mathrm{Succ}_N^r(n)|,\\
\mathsf{Steady}(N) &\Rightarrow |N| \ge r+1,\\
|N| < r+1 &\Rightarrow \mathsf{Init}(N),\\
\mathsf{Maintain} &= \{\mathsf{join},\mathsf{rectify}\}\\
&\quad \cup \{\mathsf{stabilize}\}_{CorrectChord}.
\end{aligned}$$

This is the CorrectChord stable base of Zave [6], [10].

Definition 3 (Affine replica set).

$$\begin{aligned}
\rho &\in \mathbb{N}_{>0},\\
\mathrm{rot}_i^\rho(k) &= k + \lfloor M \cdot i/\rho \rfloor,\\
\mathrm{Rep}_N^\rho(k) &= \{\mathrm{owner}_N(\mathrm{rot}_i^\rho(k)) : 0 \le i < \rho\}.
\end{aligned}$$

Storage redundancy uses affine rotations of the entry key ($\rho = 6$ by default for data topics). Successor lists remain the CorrectChord safety root for topology maintenance.

Definition 4 (Rings overlay object).

$$\mathcal{R}_N = (R, N, \mathrm{owner}_N, \mathrm{pred}_N, \mathrm{Succ}_N^r, \mathrm{Rep}_N^\rho, F, E),$$

$$\begin{aligned}
F &\subseteq N \times \mathbb{N} \times N,\\
E &= \textstyle\prod_{v \in VID} E_v,\\
\mathsf{SafetyRoot}(\mathcal{R}_N) &= (\mathrm{pred}_N, \mathrm{Succ}_N^r),\\
\mathsf{PerfWitness}(\mathcal{R}_N) &= F.
\end{aligned}$$

Proposition 1 (Rotation equivariance).

$$\begin{aligned}
t \in R, \quad N + t = \{n + t : n \in N\} \Rightarrow\\
\mathrm{owner}_{N+t}(k+t) = \mathrm{owner}_N(k) + t.
\end{aligned}$$

Proof.

$$\begin{aligned}
\forall n \in N. \quad \delta_{k+t}(n+t) &= ((n+t)-(k+t)) \bmod M\\
&= \delta_k(n).
\end{aligned}$$

□

Constraint 1 (Carrier separation).

$$\begin{aligned}\mathsf{PlacementOps} &= \{0,+,-,\delta_a,(a,b]_R\}\\ &\quad\cup\{\mathrm{owner}_N,\mathrm{Rep}_N^r\},\\ \mathsf{CryptoOps} &= \{\times,^{-1},\cdot_{scalar},\mathsf{interp}\}\\ &\quad\cup\{\mathsf{pair},\mathsf{subgroup},\mathsf{AEZ}\},\\ \mathsf{PlacementOps} &\cap \mathsf{CryptoOps}=\varnothing,\\ \mathsf{CryptoCarrier} &\in \{\mathbb{F}_q,\mathbb{G},\mathbb{Z}_q,\{0,1\}^*\}.\end{aligned}$$

Object	Carrier or law
Chord IDs	additive cyclic group $R=\mathbb{Z}/2^{160}\mathbb{Z}$
DID / Session proof	signature verifier over rings-delegation-v1
VID placement	$H_\nu : X_\nu \to R$ then owner map Ω_N
Storage merge	factored CRDT semilattice $\mathsf{E}(x) \sqcup \mathsf{R}(x)$
Store authority	crash $\times$ meaning lattice $C \times M$
Load accounting	slot floor f_T + fair pool $\mathcal{P}_T$ over $\mathcal{R}$ slots
Onion pipeline	Kleisli category $\mathbf{Kl}(M)$ with Sphinx + AEZ v5
Service receipts	dual signatures over rings-service-receipt-v1
Effects	writer monad $\mathsf{T}_A^{\mathcal{E}}$ over action lists

Table I
Carrier boundaries in the Rings 1.0 model

B. Architecture as Interfaces

Definition 5 (Layer tuple).

$$\mathcal{A}=(I,T,O,X,P,L)$$

I	$(DID,\Pi,\mathrm{Session})$
T	$(C,A_T,\mathsf{offer},\mathsf{answer},\mathsf{send},\mathsf{recv})$
O	$\mathcal{R}_N$
X	$(S_X,A_X,\mathsf{cap},I_X)$
P	$\{\mathcal{P}_\nu\}_{\nu\in Namespace}$
L	$\mathsf{App}\circ P$

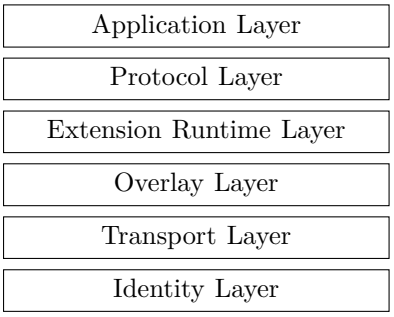


Figure 1. Layers of Rings Network

Definition 6 (Identity proof).

$$\Pi=(K,\Sigma,V,D,\mathsf{enc},\mathsf{ttl})$$

$$\begin{aligned}V &: K\times\Sigma\times B^*\to\{\top,\bot\},\\ D &: K\to R,\\ \mathsf{enc} &: \Sigma\to B^*,\\ \mathsf{ttl} &: \Sigma\to Time.\end{aligned}$$

Invariant 1 (Session authority).

$$d\in X_D,\qquad s\in K.$$

$$\mathrm{Session}(d,s,e)\Rightarrow V\big(k_d,\sigma_{d\to s},\texttt{rings-delegation-v1}\|d\|s\|e\big)=\top \\ \wedge\ now<e.$$

$$\mathsf{HopCheck}(d,s,e)\equiv\mathrm{Session}(d,s,e).$$

Definition 7 (Transport interface).

$$T=(C,\mathsf{offer},\mathsf{answer},\mathsf{send},\mathsf{recv})$$

$$A_T=\{\mathsf{Offer},\mathsf{Answer},\mathsf{Ice},\mathsf{Open},\mathsf{Send},\mathsf{Recv},\mathsf{Close}\}.$$

$$I_T : A_T^*\to \mathsf{IO}_{WebRTC}\cup\mathsf{IO}_{Native}.$$

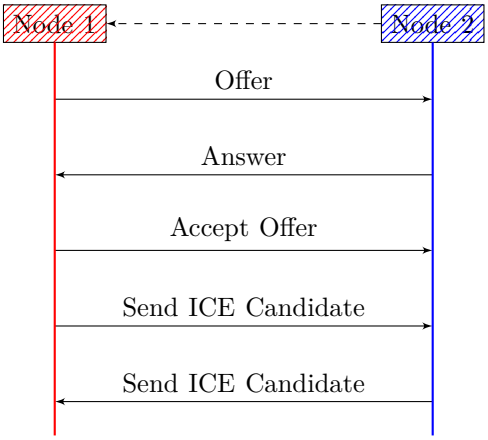


Figure 2. Standard SDP/ICE exchange

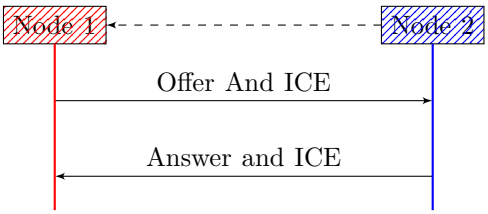


Figure 3. Rings single round-trip SDP/ICE exchange

Definition 8 (Runtime interpreter).

$$\mathcal{X}=(S_X,A_X,\mathsf{cap},I_X)$$

$$\begin{aligned}\mathsf{cap} &\subseteq A_X,\\ I_X &: A_X^*\to\mathsf{IO},\\ I_X(\epsilon) &= \mathsf{Noop},\\ I_X(\alpha\cdot\beta) &= I_X(\alpha);I_X(\beta).\end{aligned}$$

Obligation 1 (Runtime confinement).

$$\begin{aligned}\tau &: S\times I\to\mathcal{E}+(S\times O\times A_X^*),\\ \mathsf{emit}(\tau(s,i)) &\subseteq \mathsf{cap},\\ a\notin\mathsf{cap} &\Rightarrow I_X(a)=\mathsf{Reject}(a).\end{aligned}$$

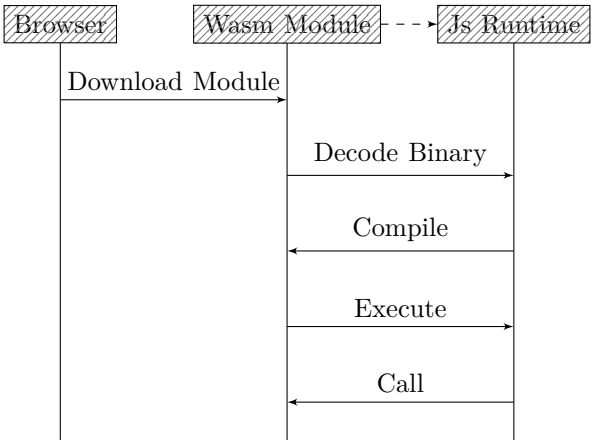


Figure 4. Wasm module execution boundary

C. Category and Effect Monad

Definition 9 (Namespace object).

$$\begin{aligned} \nu &\in \{D, V, M, H, U, Q, W\}, \\ H_\nu &: X_\nu \to R, \\ (X_\nu, H_\nu) &\in \mathbf{Set}/R. \end{aligned}$$

Definition 10 (Placement functor).

$$\begin{aligned} \Omega_N &: \mathbf{Set}/R \to \mathbf{Set}/N, \\ (X, H) &\mapsto (X, \mathrm{owner}_N \circ H). \end{aligned}$$

$$P_\nu^N = \mathrm{owner}_N \circ H_\nu : X_\nu \to N.$$

Proposition 2 (Placement functoriality).

$$f : X_\nu \to X_\mu, \qquad H_\mu \circ f = H_\nu \implies P_\mu^N \circ f = P_\nu^N.$$

Proof. $\mathrm{owner}_N \circ H_\mu \circ f = \mathrm{owner}_N \circ H_\nu$. □

Definition 11 (Writer effect monad).

$$\begin{aligned} A^* &= \mathsf{FreeMonoid}(A), \\ \mathsf{T}_A(X) &= X \times A^*, \\ \eta(x) &= (x, \epsilon), \\ (x, \alpha) \gg\!= f &= \mathsf{let}\ f(x) = (y, \beta)\ \mathsf{in}\ (y, \alpha \cdot \beta). \end{aligned}$$

Proposition 3 (Effect associativity).

$$(h^\star \circ g^\star) \circ f^\star = h^\star \circ (g^\star \circ f^\star).$$

Proof. $((\alpha \cdot \beta) \cdot \gamma) = (\alpha \cdot (\beta \cdot \gamma))$. □

Definition 12 (Typed failure stack).

$$\mathsf{T}_A^{\mathcal{E}}(X) = \mathcal{E} + (X \times A^*).$$

$$\begin{aligned} \pi_A(e \in \mathcal{E}) &= \epsilon, \\ \pi_A(x, \alpha) &= \alpha, \\ \mathsf{RecoverableReport} &\in A. \end{aligned}$$

Definition 13 (Protocol object and morphism).

$$\mathcal{P}_\nu = (X_\nu, S_\nu, I_\nu, O_\nu, A_\nu, \mathcal{E}_\nu, H_\nu, \tau_\nu),$$
$$\tau_\nu : S_\nu \times I_\nu \to \mathsf{T}_{A_\nu}^{\mathcal{E}_\nu}(S_\nu \times O_\nu).$$

A protocol morphism $\Sigma : \mathcal{P}_\nu \to \mathcal{P}_\mu = (\sigma_X, \sigma_S, \sigma_I, \sigma_O, \sigma_\mathcal{E}, \sigma_A^*)$ satisfies $H_\mu \circ \sigma_X = H_\nu$ and $\mathsf{T}_{\sigma_A}^{\sigma_\mathcal{E}}(\sigma_S \times \sigma_O) \circ \tau_\nu = \tau_\mu \circ (\sigma_S \times \sigma_I)$.

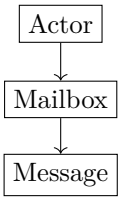


Figure 5. Actor Model

IV. Chord Implementation — PeerRing and Overlay Routing

A. Base Chord and CorrectChord Maintenance

$$\begin{aligned} \mathsf{Base} &= \mathsf{Chord}, \\ \mathsf{Maintenance} &= \mathsf{CorrectChord}, \\ \mathsf{Object} &= \mathcal{R}_N, \\ \mathsf{Witnesses} &= \{\text{Algorithms } 1, 2, 3\}. \end{aligned}$$

This section fixes Chord as the base lookup protocol and CorrectChord as the ring-safety maintenance law [4], [6], then extends the pure overlay state machine (PeerRing) with range-proved finger convergence and non-transitive WebRTC routing.

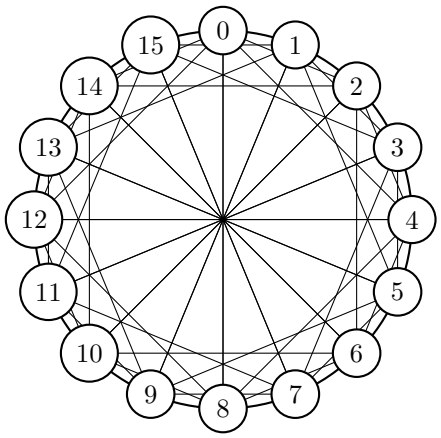


Figure 6. Chord identifier ring and finger chords

Definition 14 (Local node state and finger targets).

$$S_n = (p_n, L_n, F_n, E_n), \qquad \mathrm{target}_i(n) = n + 2^i,$$
$$\mathrm{finger}_i(n) = \mathrm{owner}_N(\mathrm{target}_i(n)),$$

where $p_n \in N \cup \{\bot\}$, $L_n \in N^{\leq r}$, $F_n : \mathbb{N} \rightharpoonup N$ with $F_n(i) \in \{\bot, \mathrm{finger}_i(n)\}$, and $E_n \subseteq E$.

Invariant 2 (Ring fixpoint and lookup owner preservation).

$$\mathrm{Inv}_{ring}(N, S) \equiv \forall n \in N.\ p_n = \mathrm{pred}_N(n) \wedge L_n = \mathrm{Succ}_N^r(n),$$

$$\mathsf{Steady}(N) \wedge \mathrm{Inv}_{ring}(N, S) \Rightarrow \mathrm{Lookup}(S, k) \Downarrow \mathrm{owner}_N(k).$$

Algorithm 1 DHT Join Transitions

```
1: function BeginJoin(n, known)
2:     pred[n] ← none; a ← FindSuccessor(n.did)
3:     return Remote(known, a)
4: function InstallSuccessor(n, s)
5:     succ[n] ← take(r, [s])
6:     return Remote(s, QuerySuccList(s))
7: function InstallSuccList(n, s, list)
8:     succ[n] ← take(r, s :: list)
9:     if head(succ[n]) ≠ none then
10:        h ← head(succ[n])
11:        return Multi(Notify(h, n.did), SyncStorage(h))
12:    else
13:        return Noop
```

Algorithm 2 DHT Lookup Algorithm

```
1: function Lookup(key, node)
2:     s ← head(succ[node])
3:     if s = none then
4:         return Repair(QuerySuccList(node))
5:     if key ∈ (node.did, s] then
6:         return Local(s)
7:     next ← closest_preceding_node(node, key)
8:     if next = none then
9:         return Repair(QuerySuccList(s))
10:    else
11:        return Remote(next, FindSuccessor(key))
```

Algorithm 3 DHT Stabilization

```
1: procedure Stabilize(n, topo)
2:     actions ← []; c ← topo.pred; s ← head(succ[n])
3:     if c ≠ none and c ∈ (n.did, s) then
4:         succ[n] ← insert(succ[n], c)
5:         actions ← actions :: FindSuccessor(c)
6:     succ[n] ← take(r, merge(succ[n], topo.succ))
7:     if head(succ[n]) ≠ none then
8:         h ← head(succ[n])
9:         actions ← actions :: Notify(h, n.did)
10:    return Multi(actions)
```

B. Range-Proved Finger Convergence

Classical Chord refreshes one random finger slot per stabilization period, requiring $\Theta(m)$ periods ($m = 160$) to converge and failing to distinguish unverified neighbor hints from proved entries [5]. Because Rings nodes include browsers whose WebRTC handshakes take seconds to minutes or fail, rings-core (dht::finger::convergence) specifies finger maintenance as a pure transition step : $(S, \text{Event}) \to (S, \text{Actions})$ built on the Chord Range Lemma and four operational disciplines.

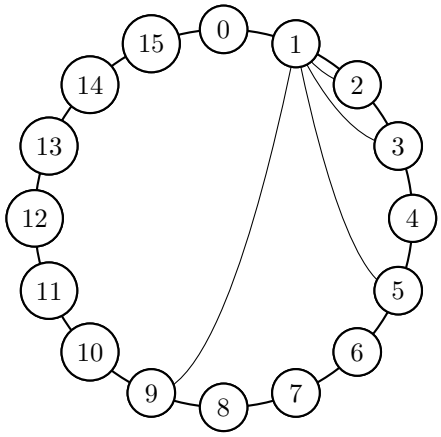


Figure 7. Chord exponential finger intervals from node 1

Lemma 1 (Chord Range Lemma [4]). For any node $n \in R$, target $x \in R$, and live membership view $V \subseteq R$,

$$\text{succ}_V(x) = y \iff \big(y \in (n, x] \ \vee \ \forall p \in (n, y) : p \notin V\big) \quad \text{with } [x, y) \cap V = \varnothing.$$

Specifically, for slot $i \in [0, L)$ ($L \le 160$) with target $t_i = n + 2^i$, let $s = \text{succ}_V(t_i)$ and $d = \delta_n(s)$. Then

$$\forall j \ge i : \quad 2^j \le d \implies \text{succ}_V(t_j) = s.$$

Proof. By definition of $\text{succ}_V(t_i)$, $[t_i, s) \cap V = \varnothing$. For any $j \ge i$ with $2^j \le d$, we have $2^i \le 2^j \le d$, so $t_j \in [t_i, s]$, and the first member of V clockwise from t_j is s. □

Definition 15 (Range proof). A lookup for slot i answered by s yields a range proof $\pi = (i, \text{id}, e_\pi, s, \text{end})$, where id is the lookup UUID [11], e_π is the evidence epoch at issue, and

$$\text{end}(i, s) = \begin{cases} L - 1, & s = n, \\ \min\big(L - 1, \lfloor \log_2 \delta_n(s) \rfloor\big), & 2^i \le \delta_n(s), \\ \mathsf{Invalid}, & \text{otherwise.} \end{cases}$$

Proof π covers the contiguous slot interval $[i, \text{end}]$.

Corollary 1 (Lookups per pass). If the true successor values partition the L slots into R maximal runs of equal successor, a complete convergence pass requires at most $R \le \min(L, |V|)$ routed lookups; under uniform identifier placement, $\mathbb{E}[R] = O(\log |V|)$.

Lemma 2 (Local Range Confirmation). Let $h = \text{head}(n)$ be the first clockwise successor of n. When n receives an authenticated stabilization report from h echoing n's active stabilization token and stating $\text{pred}(h) = n$, then

$$\forall j : \quad 2^j \le \delta_n(h) \implies \text{succ}_V(t_j) = h.$$

Applying local confirmation sets $\mathsf{ver}_j \leftarrow \top$ for all $j \le \lfloor \log_2 \delta_n(h) \rfloor$ with zero routed network lookups; if an active attempt's slot lies in that range, it is superseded via $\text{Progress}(\lfloor \log_2 \delta_n(h) \rfloor)$.

Definition 16 (Monotone Evidence Epochs). Each node n maintains an evidence state $(e, (\mathsf{chg}_j, \mathsf{ver}_j)_{j<L})$, where $e = e(n, m) \in \mathbb{N}$ is the monotone topology epoch [12], $\mathsf{chg}_j \in \mathbb{N}$ is the epoch at which hint hint_j last changed,

and $\mathsf{ver}_j \in \{\bot, \top\}$ records whether slot j holds a range-proved peer.

Law 1 (Hint change, slot-wise admission, and discontinuity). 1) Hint change: For any topology transition $H \to H'$ with changed slots $C = \{j : H_j \neq H'_j\} \neq \varnothing$, advance $e \leftarrow e+1$ and set $(\mathsf{chg}_j, \mathsf{ver}_j) \leftarrow (e, \bot)$ for all $j \in C$.
2) Slot-wise admission: Proof $\pi = (i, \mathrm{id}, e_\pi, s, \mathrm{end})$ admits slot set $A(\pi) = \{j \in [i, \mathrm{end}] : \mathsf{chg}_j \leq e_\pi\}$. If $A(\pi) = \varnothing$, π is rejected as Stale; otherwise, for every $j \in A(\pi)$, $\mathsf{hint}_j \leftarrow (s = n \ ? \ \bot : s)$ and $\mathsf{ver}_j \leftarrow \top$.
3) Discontinuity: Losing the last successor increments $e \leftarrow e+1$, clears $\mathsf{ver}_j \leftarrow \bot$ for all j, and returns the attempt owner to Idle.

Definition 17 (Single-Owner Attempt State). At most one finger lookup attempt α exists at node n:

$$\alpha \in \mathsf{Idle} \ \mid \ \mathsf{AwR}(u, p, t) \ \mid \ \mathsf{AwA}(u, p, y, e, t),$$

where $\mathsf{AwR}(u, p, t) \equiv \mathsf{AwR}((p, u), e_\rho, \tau_r)$ awaits a lookup report for slot p and request UUID u before monotonic deadline $t = \tau_r = \text{issued} + T_r$ ($T_r = 10\,\mathrm{s}$), and $\mathsf{AwA}(u, p, y, e, t) \equiv \mathsf{AwA}(\pi, \tau_a)$ awaits WebRTC transport admission of proved peer y at epoch e before lease deadline $t = \tau_a = \text{deferred} + T_a$ ($T_h = 180\,\mathrm{s}$, $T_a = T_h + 30\,\mathrm{s} = 210\,\mathrm{s}$).

Law 2 (Attempt consumption and outcomes). A report (u', p', y') consumes α iff either $\alpha = \mathsf{AwR}(u, p, t) \wedge (u', p') = (u, p)$ or $\alpha = \mathsf{AwA}(u, p, y, e, t) \wedge (u', p', y') = (u, p, y)$; all other reports are dropped as Stale without mutating state or failure streak $k \in [0, 255]$. On a consuming report, (α, c, k, ν) transitions via [13]:

$$\begin{aligned}
\mathrm{Fail}(p): &\ (\mathsf{Idle},\ \mathrm{run}(p)+1,\ k+1,\ t+f(k+1)),\\
\mathrm{Retire}(\mathrm{end}): &\ (\mathsf{Idle},\ \mathrm{end}+1,\ k+1,\ t+f(k+1)),\\
\mathrm{Progress}(\mathrm{end}): &\ (\mathsf{Idle},\ \mathrm{end}+1,\ 0,\ \bot),
\end{aligned}$$

where $\mathrm{run}(p) = \max\{j \geq p : \forall l \in [p, j],\ \mathsf{hint}_l = \mathsf{hint}_p\}$ and $f(k) = \min(2^{\min(k,6)}\,\mathrm{s},\ 60\,\mathrm{s})$ with minimum issue spacing $T_{\min} = 1\,\mathrm{s}$.

Definition 18 (Cyclic Slot Selection (next_fix_slot)). With $h = \mathrm{head}(n)$ and first routable slot $r = \lfloor \log_2 \delta_n(h) \rfloor + 1$ ($r = 0$ if $h = \bot$), let $c \in \mathbb{N}$ be the cyclic cursor, $A = \{j \geq \max(c, r) : \neg\mathsf{ver}_j\}$, and $B = \{j \in [r, \max(c, r)) : \neg\mathsf{ver}_j\}$. The selector is

$$\mathrm{next_fix_slot}(c) = \mathrm{sel}(c) = \begin{cases} \min A, & A \neq \varnothing, \\ \min B, & A = \varnothing \wedge B \neq \varnothing, \\ \bot, & \text{otherwise.} \end{cases}$$

Proposition 4 (Starvation freedom). Let U be the set of routable slots that remain unverified across a sequence of attempts. Between any two selections of $u \in U$, every other slot in U is selected at least once, and periodic revalidation reopens proved ranges whenever $\alpha = \mathsf{Idle} \wedge (\neg\exists j \geq r : \neg\mathsf{ver}_j \ \vee \ k > 0)$.

Proof. Every completed or failed attempt at slot j advances c strictly past j ($\mathrm{end}+1 > j$ or $\mathrm{run}(j)+1 > j$). Because $\mathrm{sel}(c)$ scans cyclically from c, it visits U in strict cyclic index order, preventing a single unreachable NAT peer from monopolizing the single attempt owner. □

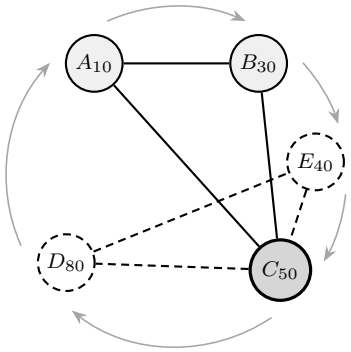


Figure 8. Interleaved ring $A(10) \to B(30) \to E(40) \to C(50) \to D(80)$ over two non-transitive WebRTC cliques $V_1 = \{A, B, C\}$ (solid) and $V_2 = \{C, D, E\}$ (dashed) sharing bridge node $C(50)$.

Law 3 (Report Claims (NotifyPredecessor / ReportFloor)). Stabilization and successor-list synchronization reports are correlated by two-phase claim tokens $\mathsf{tok} = (h, \mathrm{id}, \phi)$ with $\phi \in \{\mathsf{Req}, \mathsf{Proc}\}$. A report claims its token atomically by moving $\mathsf{Req} \to \mathsf{Proc}$, dials at most $K+1$ distinct candidates for stabilization (and at most K for successor sync) re-checking $\phi = \mathsf{Proc}$ before each dial, and clears tok on completion or on any SuccessorHeadChanged transition (ReportFloor).

C. Non-Transitive WebRTC Overlay and Bridge-Delegated Delivery

In browser and native WebRTC deployments, symmetric NATs, enterprise firewalls, and disjoint LANs violate underlay transitivity [7], [14]: if node a links to b and b links to c, a may still fail ICE negotiation with c. Figure 8 illustrates two full-mesh WebRTC cliques $V_1 = \{A, B, C\}$ and $V_2 = \{C, D, E\}$ joined at an articulation (bridge) node $C(50)$ with interleaved ring order $A(10) \prec B(30) \prec E(40) \prec C(50) \prec D(80)$.

Rather than injecting unstructured neighborhood gossip or virtual fingers [15], [16] (which would violate the direct-link invariant $\mathrm{Referenced}(S_n) \subseteq \mathrm{Admitted}(n)$), Rings resolves non-transitive WebRTC routing with zero extra control frames via four mechanisms:

Definition 19 (Extended Topology Carrier and Multi-Predecessors). Each node $n \in V$ maintains

$$S_n = \big(n,\ \mathrm{succ}_n,\ \mathrm{pred}_n,\ \mathrm{Preds}_K(n),\ \mathrm{fingers}_n,\ \mathrm{bridged}_n,\ \Phi_n,\ \Pi_n\big),$$

where $\mathrm{Preds}_K(n) = \langle p_1, \ldots, p_m \rangle$ ($m \leq K$, supporting capacity $K = 4$) is ordered by descending clockwise distance $\delta_n(p_1) > \cdots > \delta_n(p_m)$ (ascending counter-clockwise distance $\delta_{p_i}(n)$), with primary predecessor $\mathrm{pred}_n = p_1$. All referenced peers satisfy

$$\mathrm{Referenced}(S_n) = \big(\mathrm{set}(\mathrm{succ}_n) \cup \mathrm{set}(\mathrm{Preds}_K(n)) \cup \mathrm{ran}(\mathrm{fingers}_n)\big) \setminus \{n\} \subseteq \mathrm{Admitted}(n).$$

Law 4 (Multi-Predecessor Rectification and Candidate Dial Backoff). Empty-confirmed commit: When $r = \text{head}(n)$ remains routable ($\text{Routable}(n, r)$), n commits $\text{Stabilize}(r)$ and emits $\text{Notify}(r)$ even if confirmation filtering leaves confirmed empty.

2) Candidate dial backoff (DialBackoff): Failed or expired candidate handshakes enter a bounded negative cache $\mathcal{B}_n$ ($B_{\max} = 64$) with exponential cooldown $\beta(f) = \min(\beta_0 \cdot 2^{\min(f-1,4)}, \beta_{\max})$ (configurable from base $\beta_0 = 5\,\text{s}$ and cap $\beta_{\max} = 60\,\text{s}$ up to 30–300 s), suppressing redundant SDP offers while allowing explicit application connects or successful admission to clear $\mathcal{B}_n(c)$ immediately.
3) Supersede-on-Closer, Retain-on-Sub-Ring (rectify_preds): Given geometrically admissible candidate c and primary predecessor p_1: if $p_1 = \bot$ or $\delta_n(c) > \delta_n(p_1)$ (c is strictly closer to n), $\text{Preds}_K(n) \leftarrow \langle c \rangle$; if $\delta_n(c) \le \delta_n(p_1)$ (notification from a farther sub-clique predecessor), insert c into $\text{Preds}_K(n)$ in descending $\delta_n(\cdot)$ order, deduplicate, and truncate to K.

Lemma 3 (Articulation Retention and Sub-Clique Local Finger Proof). On any transitive ring, $\text{Preds}_K(n)$ collapses to the singleton global predecessor $[\text{pred}_V(n)]$. At an articulation node C joining $m \le K$ cliques $V_1, \ldots, V_m$, $\text{Preds}_K(C)$ retains all m sub-clique predecessors $p_i = \text{pred}_{V_i}(C)$ within two stabilization rounds, protecting every bridge link (p_i, C) from eviction and allowing each p_i to filter reported.preds to confirmed.pred = Some(p_i) and prove its local finger range $[0, \lfloor \log_2 \delta_{p_i}(C) \rfloor]$ via Lemma 2.

Law 5 (Passive 1-Hop Gap-Bridge Hints and Bridge-Delegated Delivery). Whenever n processes an authenticated TopoInfo report from $r \in \text{succ}_n$, the unconfirmed peers $U(n, r) = (\text{set}(\text{reported.preds}) \cup \text{set}(\text{reported.succ})) \setminus \{p : \rho_n(p)\}$ update the zero-TTL partial map $\text{bridged}_n : \mathsf{Did} \rightharpoonup \mathsf{Did}$ ($|\text{bridged}_n| \le 2K^2$, with $p \mapsto r$ iff $r \in \text{Referenced}(S_n) \wedge p \notin \text{Referenced}(S_n)$). Payload delivery carries $\sigma \in \text{RouteStage} = \text{aim}(\mathsf{Did}_\bot) \times \mathsf{bridged}(\mathbb{B})$ (fields $(\text{via}, \text{handed_off})$ with $\text{aim}(\sigma, T) = \text{via.unwrap_or}(T)$):

- Rule B1 (1-Hop Bridge Target Forwarding): If $\neg\,\text{Linked}_n(T)$, $\sigma.\text{via} = \bot$, and $\text{bridged}_n(T) = \text{Some}(b)$ with $\text{Linked}_n(b)$, forward to (b, σ) when $\delta_n(b) < \delta_n(T)$ and to $(b, (\bot, \top))$ when $\delta_n(b) > \delta_n(T) \wedge \neg\sigma.\text{handed_off}$.
- Rule B2 (Gap-Bridge Handoff): When greedy clockwise routing toward $a = \text{aim}(\sigma, T)$ hits a local minimum ($\text{Handoff}(h)$) with $\neg\sigma.\text{handed_off} \wedge \sigma.\text{via} = \bot$, if $\exists (p \mapsto b) \in \text{bridged}_n$ with $\text{Linked}_n(b) \wedge \delta_n(a) \le \delta_n(p) < \delta_n(h)$, delegate across the gap via $(b^*, (\bot, \top))$ minimizing $\delta_n(p)$.

Theorem 1 (Bounded Single-Hop Bridge Delegation and Route Safety). Without source-routing hop lists on the wire, delivery_step over $\text{RouteStage} = \text{aim}(\mathsf{Did}_\bot) \times \mathsf{bridged}(\mathbb{B})$ preserves all four route-safety invariants on any underlay graph $G = (V, L)$: (1) Linked ($\{n_k, n_{k+1}\} \in L$), (2) NeverOvershoots ($\delta_{n_k}(n_{k+1}) < \delta_{n_k}(\text{aim})$ on every non-crossing hop), (3) SingleCrossing and Acyclic (the boolean crossing bit transitions $\bot \to \top$ at most once, visiting no (n_k, n_k)) twice and bounding every route to $\le 2|V|$ hops), and (4) all-pairs delivery in ≤ 2 hops (≤ 3 hops pre-sync) across bridge-connected cliques of size $\le K$.

Proof. Within each fixed stage σ, $\delta_{n_k}(\text{aim}(\sigma, T)) \in \mathbb{N}$ strictly decreases on every non-crossing hop, visiting at most $|V| - 1$ nodes. Rules B1 (second branch) and B2 flip handed_off : $\bot \to \top$ and cannot fire when handed_off = $\top$, so at most one ring-wrap or gap-bridge crossing occurs per route, yielding $\le 2|V|$ hops total. In a two-clique graph $V_1 \cup V_2$ sharing bridge C, Lemma 3 places all of $(V_1 \cup V_2) \setminus \{C\}$ into C's TopoInfo, so every $u \in V_i \setminus \{C\}$ learns $\text{bridged}_u(v) = \text{Some}(C)$ for $v \in V_j \setminus \{C\}$ and delivers via $u \to C \to v$ in 2 hops. □

V. CRDT Storage and Removal Stability

A. DID, VID, and Replicated Entry Algebra

Definition 20 (DID and VID namespaces).

$$\mathcal{D} = (X_D, H_D, \Pi, \text{Session}), \qquad \mathcal{V} =$$
$$H_D : X_D \to R, \qquad H_V :$$
$$\text{PrivateKey}(H_V(x)) = \bot, \qquad \text{Auth}_V(op, x, \sigma, caps) \in$$

Definition 21 (Biased circular order).

$$A \le_C B \iff \delta_C(A) \le \delta_C(B).$$

Algorithm 4 Biased Circular Comparison

```
function BiasedCompare(A, B, C)
    a ← δ_C(A); b ← δ_C(B)
    if a < b then return A <_C B
    else if a > b then return B <_C A
    else return A = B
```

Definition 22 (Replicated entry algebra and placement invariant).

$$\mathcal{E}_v^{join} = (E_v, \sqsubseteq, \sqcup, \bot), \qquad \mathcal{E}_v^{final} = (E_v, \mathsf{conflict}_v, \mathsf{finalize}_v),$$

where $(E_v, \sqsubseteq, \sqcup, \bot)$ is a join-semilattice [17] ($x \sqcup y = y \sqcup x$, $(x \sqcup y) \sqcup z = x \sqcup (y \sqcup z)$, $x \sqcup x = x$, $x \sqcup \bot = x$), and placement satisfies

$$\text{Inv}_{place} \equiv \forall \nu, x.\ \text{replicas}(H_\nu(x)) \subseteq \text{Rep}_N^\rho(H_\nu(x)).$$

Definition 23 (Mailbox and hidden service descriptor).

$$mbox(d, e) = H_M(\texttt{rings.mailbox} \| d \| e),$$
$$entry_M = (\eta, exp, proof_s, dst, cipher),$$
$$h = (\nu, \rho, Intro, Relay, Cap, K, exp, rank),$$
$$VID(h) = H_H(h), \qquad \mathsf{Lookup}(h) = P_H^N(h).$$

B. Carrier State, HLC Timestamps, and Cryptographic Data Provenance

Definition 24 (Data Carrier and HLC Timestamps). A Rings data-topic carrier (dht::entry, crdt.rs) is a 4-tuple

$$x = (V, D, \rho, \beta),$$

where V : Bytes $\rightharpoonup$ Dot maps live elements to addition dots, D : Digest $\rightharpoonup$ Dot maps Keccak-256 element digests

$h(v)$ to removal dots (tombstones), $\rho \in \mathrm{Ver}_\perp$ is the overwrite floor register, and $\beta \in \mathbb{N}_\perp$ is the carrier liveness bound. Each version/dot carries a Hybrid Logical Clock (HLC) timestamp

$$\tau_{\mathrm{HLC}} = (w, c, o),$$

ordered lexicographically by wall-clock time $w = \tau(d)$, logical counter $c \in \mathbb{N}$, and writer/origin DID $o \in R$. A removal $(e, r) \in D$ covers an addition $(v, d) \in V$ iff $h(v) = e \wedge d \le r$. A carrier x is normalized iff no addition in V is covered by D or $\le \rho$, and no removal in D is $\le \rho$. The join $x \sqcup y$ is pointwise max followed by normalization.

Law 6 (Per-Writer Cryptographic Provenance (DataProvenance)). Every element addition (Append / Extend, #1179) and removal tombstone (Tombstone, #1220) carries a per-writer cryptographic provenance proof $\pi_{\mathrm{prov}} \in \mathsf{DataProvenance}$ signed over the canonical domain separator `rings-data-carrier-v1\0`:

$$\mathsf{Verify}\big(k_o, \sigma_o, \texttt{rings-data-carrier-v1\textbackslash 0} \\ \|\mathsf{op}\|k_{\mathrm{topic}}\|e\|(w, c, o)\big) = \top,$$

where $\mathsf{op} \in \{\mathsf{Add}, \mathsf{Remove}\}$, $k_{\mathrm{topic}} \in R$ is the placement key, $e = h(v)$ is the element digest, and $o = D(k_o)$ matches the HLC origin DID. Unauthenticated deltas or clock skews exceeding $c + \sigma$ are rejected at admission.

C. Removal Stability Horizon, Factored Carrier, and Tombstone Bounds

Definition 25 (Stability Horizon and Retirement). With element retention horizon $H = 86\,400\,\mathrm{s}$ (configurable down to $6 \cdot 10^6$ ms on fast-expiring topics) and clock-skew tolerance $\sigma = 300\,\mathrm{s}$ (3 000 ms under tight NTP), a removal (e, r) (or floor ρ) is stable at clock t iff

$$t \ge \tau(r) + H + \sigma.$$

The retirement projection $\mathrm{retire}_t(x)$ drops additions with $t \ge \tau(d) + H$ and removals/registers with $t \ge \tau(r) + H + \sigma$. retire_t is a join-semilattice homomorphism satisfying $\mathrm{retire}_t \circ \mathrm{retire}_s = \mathrm{retire}_{\max(s,t)}$.

Lemma 4 (Stable removes cover only retired adds). If (e, r) is stable at the clock t_p of some node p, then every addition (v, d) covered by (e, r) is already retired at the clock t_q of every node q at the same instant ($t_q \ge t_p - \sigma \ge \tau(r) + H \ge \tau(d) + H$).

Law 7 (Remove survival and Factored Carrier). An unstable remove or register leaves a holder's replica only by (i) retire_t at stability, (ii) an ack-gated range handoff whose receiver holds it, or (iii) a platform loss event—never by storage eviction, cap, or truncation. To enforce this, every normalized carrier x factors into an element side and a remove side:

$$x = \mathsf{E}(x) \sqcup \mathsf{R}(x),$$
$$\mathsf{E}(x) = (V, \varnothing, \perp, \beta), \qquad \mathsf{R}(x) = (\varnothing, D, \rho, 0).$$

$\mathsf{E}(x)$ is stored in the data store at authority $(\mathsf{Cch}, \mathsf{Reb})$, whereas $\mathsf{R}(x)$ is persisted in the dedicated removal store (dht-removals) at $(\mathsf{Dur}, \mathsf{Ret})$ (RecordAuthority::Retained). Byte-budget eviction maps $x \mapsto \mathsf{R}(x)$ (dropping only $\mathsf{E}(x)$), and $\mathrm{put}(k, x)$ flushes $\mathsf{R}(x)$ before writing $\mathsf{E}(x)$.

Theorem 2 (No Resurrection). Under Law 7, if holder p holds (e, r) at t_0 and experiences no loss event before $\tau(r) + H + \sigma$, then across any sequence of joins, evictions, and crashes, no read at p after t_0 ever serves an addition covered by (e, r).

Proof. Before stability ($t < \tau(r) + H + \sigma$), Law 7 keeps $D_p(e) \ge r$ in the durable removal store across evictions and crashes, so $\mathrm{get}(k) = \mathsf{E}_k \sqcup \mathsf{R}_k$ normalizes away any covered addition. At and after stability, Lemma 4 guarantees that every covered addition has $t \ge \tau(d) + H$ and is stripped by retire_t. □

Law 8 (Tombstone Bound (Storage Tombstone Full)). Each data carrier is bounded by $T_{\max} = D_{\max} = \texttt{ENTRY_DATA_MAX_LEN} = 1024$ removes. Any peer delta δ with $|D_\delta| > T_{\max}$ is inadmissible, and any operation or join whose retired result $y = \mathrm{retire}_t(x \sqcup \delta)$ has $|D_y| > T_{\max}$ is refused with $\mathsf{Err}(\mathsf{Exhausted})$ (Storage Tombstone Full), leaving the stored carrier unchanged. Every admitted carrier encodes in at most $W = w_0 + D_{\max}(P_{\max} + \lambda + w_D) + T_{\max} w_T + w_\rho \approx 33.83\,\mathrm{MB} \le \texttt{TRANSPORT_MAX_SIZE}$ (60 MB), and any admissible sync delta simulates a sequence of $\le 2|D_\delta| + |V_\delta| + 1$ writer operations.

D. Fetch Cache Linearization and Signed Lease Heartbeats

Law 9 (Fetch Cache Pure-Read Linearization (Law 6)). The local fetch cache $c : K \rightharpoonup X$ is a $(\mathsf{Vol}, \mathsf{Reb})$ mirror whose mutations (put, obs, evict) execute atomically under the cache lock while reads are pure projections $t \mapsto \mathrm{retire}_t(c[k])$ that perform zero store writes or deletions. Consequently, every cache transition linearizes [18] at its store put and every read at its store get, eliminating concurrent read-retire lost-update races (#922).

Law 10 (Signed Lease Heartbeats (LeaseHeartbeat)). To eliminate the $\approx$ 200 tombstones and $O(N^2)$ read traffic generated when service registrants overwrite timestamped descriptors every $I = 30\,\mathrm{s}$ (#918), a registrant signs a fixed lease descriptor d over domain `rings-lease-heartbeat-v1\0` binding its identity, process epoch, capabilities, freshness window $T = 90\,\mathrm{s}$, and lease expiration $\ell = t + \Lambda$ ($\Lambda = 10\,\mathrm{min}$). Each heartbeat at clock t emits only Extend($[d]$) (writing zero tombstones and performing zero registry reads), rotating $d \mapsto d'$ only when $t + 2I \ge \ell$ or capabilities change. Readers at clock t accept the verified descriptor of greatest ℓ satisfying $\ell \le t + \Lambda + \sigma$, $T \le \Lambda$, $t \le \ell$, and $t \le \tau^*(d) + T + \sigma$.

VI. Durable Local Stores and Replicated Custody

A. Authority Lattice, Presence Markers, and Platform Crash Durability

A Rings node persists replay windows, onion entry guards, removal records, and relay mail across native

Table II
Authority assignments across Rings 1.0 local stores

Local Store	Authority (c, m)	Discharged Obligations
DHT Data Store	$(\mathsf{Cch}, \mathsf{Reb})$	Disposable / rebuildable
Fetch Cache / Measure	$(\mathsf{Vol}, \mathsf{Reb})$	Pure-read mirror
Onion Entry Guards	$(\mathsf{Dur}, \mathsf{Reb})$	A, D (DurableCache)
CRDT Removal Store	$(\mathsf{Dur}, \mathsf{Ret})$	A, D, N (Retained)
Relay Custody Carriers	$(\mathsf{Dur}, \mathsf{Ret})$	A, D, N (r-replicated on Dur°)
Transaction Replay Store	$(\mathsf{Dur}, \mathsf{Aut})$	A, D, N, W, F (Authoritative)

file systems and browser IndexedDB backends. We unify their presence and crash-survival guarantees via a product authority lattice (RecordAuthority) [19].

Definition 26 (Crash-Meaning Authority Lattice $C \times M$). The crash chain $C = \mathsf{Vol} \sqsubset \mathsf{Cch} \sqsubset \mathsf{Dur}$ and meaning chain $M = \mathsf{Reb} \sqsubset \mathsf{Ret} \sqsubset \mathsf{Aut}$ form the componentwise lattice $C \times M$ with monotone obligation map $O(c, m) = O(c) \cup O(m)$:

$$O(\mathsf{Vol}) = O(\mathsf{Cch}) = \varnothing,$$
$$O(\mathsf{Dur}) = \{\text{Atomicity (A), Survival (D)}\},$$
$$O(\mathsf{Reb}) = \varnothing, \qquad O(\mathsf{Ret}) = \{\text{No-Eviction (N)}\},$$
$$O(\mathsf{Aut}) = \{\text{N, Witnessed-Presence (W), Fail-Closed (F)}\}.$$

By the Admissibility Law, a store may open at (c, m) only if $c \sqsupseteq \mathrm{req}(m)$, where $\mathrm{req}(\mathsf{Reb}) = \mathsf{Vol}$ and $\mathrm{req}(\mathsf{Ret}) = \mathrm{req}(\mathsf{Aut}) = \mathsf{Dur}$.

Law 11 (Presence Markers (.p) and Per-Platform Crash Durability).
1) Presence markers (.p): In an authoritative native store, each record $n = \mathrm{hex}(\mathrm{SHA1}(k))$ has an empty witness marker n.`p` created and directory-synced only after n.`tmp` $\to n$ is flushed, and removed before n on deletion, maintaining invariant $P \subseteq R$. At restore, any $n \in P \setminus R$ is classified as $\mathsf{Missing}(n)$ and fails stream n closed (discharging W for every partial loss λ_{part}, whereas total directory loss λ_{tot} is provably undetectable by any in-domain witness and serves as the documented operator reset).
2) macOS / iOS: File data and directory entries are flushed via `fcntl(F_FULLFSYNC)` [20].
3) Windows: Temporary files flush via `FlushFileBuffers` and all directory mutations execute via `MoveFileExW` with `MOVEFILE_REPLACE_EXISTING | MOVEFILE_WRITE_THROUGH` [21].
4) Linux / Unix: Data files flush via `fsync` followed by parent directory `fsync` after atomic rename [19].
5) Browsers (IndexedDB): Read-write transactions at Dur open with explicit $\{\textit{durability} : \text{"}\texttt{strict}\text{"}\}$ (forcing OS buffer flush / SQLite EXTRA or full WAL checkpoint across Chromium, Gecko, and WebKit [22]–[26]) combined with `navigator.storage.persist()` [27]. Because browsers delete origin storage whole ($P = \varnothing$) and ephemeral contexts can only claim durability (Dur°), node-local stores open at (Dur°, m) with session end as λ_{tot}, while custody of other nodes' mail requires $r \geq 2$ replicated holders.

B. Group Commit and Link-Local Volatile Replay Windows

Law 12 (Group Commit (put*)). Replay transitions $t_1 < t_2 < \ldots$ execute in lock order on speculative tables $\hat{T}$ with undo log entries u_i and coalesce into batches B_j written by a single put$^*(W_j)$: on native targets, all n.`tmp` files are pushed before a single drive-cache barrier (`F_FULLFSYNC`), renamed, and directory-synced once; on IndexedDB, all puts share one strict read-write transaction. Callers are released only after put$^*(W_j) = \mathsf{Ok}$ (latency $\leq 2\phi$); on Err, $\hat{T}$ rolls back via u_i to the last durable state T.

Law 13 (Link-Local Volatile Replay Windows ($\mathcal{W}_{s,g}$)). Direct neighbor control traffic (Join|Sync|Leave|Send) and application namespaces registered in D_r as delegating replay (such as Sphinx onion cells, which enforce at-most-once admission in their own Bloom filter R_i and process epoch e_i) are classified by $\mathrm{ll}(t, c)$ on authenticated connection c of live generation g and admitted against an in-memory per-generation window $\mathcal{W}_{s,g}$ instead of the durable store $\mathcal{W}_s$. This eliminates durable disk flushes on the onion/control hot path while preserving per-generation duplicate/fork detection and cross-restart replay immunity.

C. Receiver Custody Set and Age-Gated Replicated Relay Custody

Definition 27 (Receiver Custody Set ($\mathcal{K}_s$)). To prevent relay mail admitted into $\mathcal{W}_s$ from being lost if the node restarts before the application consumes it from the volatile inbox (#936), the receiver record of stream s co-persists $(\mathcal{W}_s, \mathcal{K}_s)$ atomically in a single $(\mathsf{Dur}, \mathsf{Aut})$ record, where $\mathcal{K}_s \subseteq \mathbb{N} \times \mathrm{Digest} \times \mathbb{N}$ holds admitted but untaken deliveries (q, h, x) with spent horizon $x = ts + ttl + H_I + 2\epsilon$ ($|\bigcup_s \mathcal{K}_s| \leq$ `RELAY_INBOX_MAX_LEN` $= 64$).

Law 14 (Custody Lifecycle and Age-Gated Replicated Relay Chain).
1) Custody drain and release: On drain after put*, an element (s, q, h) already in $\mathcal{K}_s$ is requeued without re-checking $\mathcal{W}_s$; a new element commits $(\mathcal{W}'_s, \mathcal{K}_s \cup \{(q, h, x)\})$ before queueing; and the carrier tombstone/Ack is issued only after the application takes the message and commits $(\mathcal{W}_s, \mathcal{K}_s \setminus \{(q, h, x)\})$. By information-theoretic indistinguishability, no bounded-window receiver can safely admit uncustodied mail that has fallen behind live traffic ($q < \mathrm{rmin}(\mathcal{W}_s)$).
2) Age-gated replicated custody ($\mathcal{M}_r(k)$): Because browser holders operate at claimed durability Dur° and experience high churn [28], a node advertises relay custody only after surviving an online age gate $A_{\min}$ ($A_{\min} = 30\,\text{min}$ in production profiles, or ≥ 4 stabilization periods). Each mail key k is replicated across a chain $\mathcal{M}_r(k)$ of the first r age-gated holders clockwise from $\mathrm{owner}_N(k)$ ($r = 4$ to $r = 8$).
3) Chain replication and digest repair: Holds forward head-to-tail along $\mathcal{M}_r(k)$ (digest first, payload on miss) and acknowledge tail-to-head only after all $q(k) \leq r$

members persist the element. Adjacent chain members probe liveness every $P_c = 2.5\,\mathrm{s}$ (suspect after $\tau_c = 7.5\,\mathrm{s}$) and repair via bundled carrier digest-set comparison on membership change or periodic pass, restoring r copies within $D \le T_d + r\delta$.

VII. Fair Admission, Slot Accounting, and Load Bounds

A. Cleanest-First Slot Roster and Fair Slot Accounting

Because a Rings DID is a free public-key digest, any node-wide table keyed solely by origin DID can be exhausted by one adversary minting thousands of identities (#907, #1060). Rings anchors resource accounting to the authenticated transport connection slot.

Definition 28 (Cleanest-First Slot Roster (SlotRoster)). A node's connection registry holds at most

$$\mathcal{R}(s) = 2(F + s + 1)$$

active neighbour edges, where $F = 160$ is the finger table size and s is the successor capacity ($\mathcal{R}(4) = 330$ at $s = 4$; $\mathcal{R}(3) = 328$ at $s = 3$). The SlotRoster indexes slots $\kappa \in \{0, \dots, \mathcal{R}(s) - 1\}$, tracking per-slot metadata $(\mathit{epoch}, \mathit{strikes}, \mathit{last_seen_ms})$ and live floor utilization across shared tables T:

$$w_t(\kappa) = \max_T \frac{\min\big(\mathrm{use}^t_{T,\kappa},\, f_T\big)}{f_T}.$$

When a new edge e is admitted, it is assigned the cleanest free slot $\kappa(e) = \arg\min_{\kappa \in F_t} w_t(\kappa)$ (ties broken by least index), and charges in table T remain bound to κ until they expire by T's own horizon law.

Law 15 (Slot Accounting). Every shared table T is parameterized by a per-slot guaranteed floor f_T, a shared overflow pool $\mathcal{P}_T$, a local share L_T for principal λ, and dedicated shares S_d for $d \in D_T$, yielding the hard node-wide bound

$$B_T(s) = \mathcal{R}(s)\, f_T + \mathcal{P}_T + L_T + \sum_{d \in D_T} S_d.$$

With slot usage use_κ, pool overflow $o_\kappa = \max(0, \mathrm{use}_\kappa - f_T)$, and active claimant set $\mathcal{A}(t) = \{\kappa : o_\kappa > 0 \text{ or } \kappa \text{ requests pool overflow}\}$ of size $n(\mathcal{A}(t)) = |\mathcal{A}(t)| \ge 1$, an item introduced by slot κ is admitted iff either:

1) $\mathrm{use}_\kappa < f_T$ (within the slot's guaranteed floor), or
2) $\sum_{\kappa'} o_{\kappa'} < \mathcal{P}_T$ and $o_\kappa < \lceil \mathcal{P}_T / n(\mathcal{A}(t)) \rceil$ (within the slot's fair pool claim).

Theorem 3 (Multi-Table Slot Isolation). Across all five shared node tables—(1) Application Inbox / Mailbox Ledger, (2) Relay Store, (3) CRDT Removal Store, (4) Transaction Replay Table, and (5) Chunk Reassembly Buffer O—Law 15 and Definition 28 guarantee:

1) $|T| \le B_T(s)$ in all reachable states;
2) any edge holding slot κ with $\mathrm{use}_\kappa < f_T$ is never refused, regardless of how many origins, edges, or slots other parties control;
3) an over-floor slot is guaranteed at least $f_T + \lceil \mathcal{P}_T / \mathcal{R}(s) \rceil$ items as prior claimants expire;
4) under cleanest-first assignment (SlotRoster), an adversary rotating Sybil connections cannot degrade a newcomer's floor with a single departure-and-rejoin (unlike least-free-index selection), requiring $\mathcal{R}(s) - c$ distinct slot admissions within one horizon h to touch every free slot.

Proof. Items (1)–(3) follow because floor checks read use_κ alone, departures never erase slot charges, and claimants above $\lceil \mathcal{P}_T / n \rceil$ cannot grow while prior items drain. Item (4) holds because a free slot with $w_t(\kappa) = 0$ is always selected before any partially charged slot κ' with $w_t(\kappa') > 0$. □

B. Replay Stream Eviction and Credit Reclaim

Law 16 (Replay Stream Spent-Horizon Eviction). A receiver replay stream s with maximum admitted transaction expiry $x_s = \max(ts + ttl)$ is spent at clock t once $t \ge x_s + \iota + \sigma$ (for default control/storage/E2E traffic, the spent horizon is $H_{\mathrm{spent}} = 600\,\mathrm{s}$; for offline-capable application streams, $\iota = H_I + \sigma$ where $H_I = 7\,\mathrm{d}$). Because transactions past $x_s + \iota + \sigma$ are rejected by envelope TTL and relay hold-horizon checks prior to replay lookup, spent streams holding no custody $\mathcal{K}_s$ are lazily evicted without reopening replay. With $f = 32$, $\mathcal{P} = 3\,328$, $L = 1\,024$, and inbox share $S_\mu = 768$, no honest slot can be permanently locked out.

Law 17 (Credit Reclaim (Credit/Shrink/Returned) and Hard Frame Bound). Each edge has four lanes ℓ (one priority control lane with floor $g_{\mathrm{prio}} = W = 16$, and three data lanes with floor $g_\ell = 1$ and cap $W = 16$). To bound total in-flight and buffered frames node-wide without dropping frames, credit frames carry a lane generation g:

- $\mathsf{Credit}(A, g)$: receiver grants limit A in generation g;
- $\mathsf{Shrink}(L', g)$: receiver reclaims unused pooled credit on an idle ($T_{\mathrm{idle}} = 1\,\mathrm{s}$) or over-share lane down to $L' = \min(R + g_\ell, A)$;
- $\mathsf{Returned}(B, g)$: sender commits at most $B = \max(c, L')$ frames and ignores older generations.

With frame pool $\mathcal{P}_F = 256$ frames (16 MiB), total node frame liability satisfies the hard bound

$$\sum_{e,\ell} \mathrm{liab}(e, \ell) \;\le\; B_F(s) = c(s) + W_{\mathrm{shrink}} = \mathcal{R}(s)(W+3) + \mathcal{P}_F$$

(6 488 frames at $s = 3$; 6 526 at $s = 4$), with zero honest frame drops and guaranteed per-edge pooled throughput $\ge \min(3(W-1), \lfloor \mathcal{P}_F / n \rfloor)$.

C. Single-Charge Chunk Reassembly and Per-Category Barrier

Law 18 (Single-Charge Chunk Reassembly and Credit--Time Rate Floor). A chunked message $[0, n]$ of chunk size d_0 reserves its full footprint $m = 2nd_0$ once on its

first chunk in both the mailbox ledger and the open-message table O ($f_O = 256\,\mathrm{KiB}$, $\mathcal{P}_O = 128\,\mathrm{MiB}$, $B_O(s) = \mathcal{R}(s)f_O + \mathcal{P}_O$), waiting in ResourceOrderedQueue order ($O((g+k+r)\log n)$ per event) and moving into dispatch at completion without re-charging. To prevent slowloris reservation pinning without penalizing receiver-throttled links, the receiver tracks credit-time $\hat{\tau}$, which advances over $[t_k, t_{k+1})$ by $\min(t_{k+1} - t_k,\, c_k d_0/\rho)$ only while the sender holds $c_k = A - N > 0$ unused credits. Messages must maintain a streaming rate floor ($r_{\min} = 1024\,\mathrm{B/s}$ to $\rho = 10\,\mathrm{B/ms}$) with grace window $G = 15\,\mathrm{s}$ (up to $(W+1)d_0/\rho + \Delta$), expiring attributably if received bytes fall below $b < \rho\hat{\tau} - Wd_0$.

Law 19 (Per-Category Reassembly Barrier (Control | Interactive | Bulk)). Inbound dispatch maintains independent ordering barriers per traffic category $\kappa \in \{\mathsf{Control}, \mathsf{Interactive}, \mathsf{Bulk}\}$ (mapped to protocol lanes $\{ctl, sto, e2e, app\}$): a frame f of category κ waits only behind unfinished reassembly events $u \in U$ with $seq(u) < seq(f) \wedge \kappa(u) = \kappa$. This guarantees exact sequence order within every category (including chunked control frames) while preventing bulk data reassembly from blocking control or interactive lanes (#927).

VIII. Privacy Layer — Cryptographic Carriers and Sphinx Onion Kleisli Pipeline

A. Separate Cryptographic Carriers

Definition 29 (End-to-end carrier, signed frame, and secret sharing). End-to-end encryption and secret sharing operate over groups $\mathbb{G}$ of prime order q ($\mathbb{G} \neq R$, Algorithm 5) and finite fields $\mathbb{F}_q[X]$ [29]:

$$f = (sid, seq, last, cipher, pk_s, \sigma_s),$$

$$\mathrm{FrameValid}(f) \iff V(pk_s, \sigma_s, m_f) = \top \wedge \mathrm{Session}(D(pk_s)) = sid,$$

$$P \in \mathbb{F}_q[X], \quad sh_i = (i, P(i)) \in \mathbb{F}_q^2, \quad \mathrm{reconstruct}(\{sh_i\}) = P(0).$$

Algorithm 5 ElGamal Encryption and Decryption on $\mathbb{G}$

```
procedure Encryption(m, h = g^x, g)
    Sample k ∈ Z_q; return (c_1 = g^k,  c_2 = m · h^k)
procedure Decryption(c_1, c_2, x)
    return m = c_2 · (c_1)^{-x}
```

Algorithm 6 End-to-End Chunking Algorithm

```
procedure E2E Chunking(data, chunk_size)
    chunks ← []; total ← ⌈len(data)/chunk_size⌉
    for i ∈ [0, total) do
        append data[i · chunk_size : (i + 1) · chunk_size] to chunks
    return chunks
```

Invariant 3 (Chunk reassembly).

$$\mathrm{complete}(m) \iff \forall i < total(m).\ \exists! c_i \wedge \sum_i |c_i| \leq B_m \wedge now < ttl(m).$$

B. Operation Signature and Kleisli Pipeline Composition

Rings 1.0 replaces legacy relay framing with the Sphinx Onion Kleisli Pipeline (rings-node::onion, #834), generalizing onion circuits from simple exit tunnels to client-sealed loops over a static operation signature Σ composed in the Kleisli category $\mathbf{Kl}(M)$ [30]–[36].

Definition 30 (Operation Signature Σ and Partial Σ-Algebras (D1′, D2)). The static shape signature is

$$\Sigma = \{\mathsf{relay}, \mathsf{tcp}, \mathsf{https}\} \cup \Sigma_{\mathsf{invoke}},$$

where $\Sigma_{\mathsf{invoke}} = \{\mathsf{invoke}\langle \mathrm{In}, \mathrm{Out}, W, L\rangle : W \in \mathcal{W},\ L \in \mathcal{L}\}$ spans discrete output-width classes $W(f)$ and latency classes $L(f)$, and invoke backends $\omega \in \Omega$ include the core distributed operations:

dht.find_successor, dht.notify, dht.sync, storage.lookup, storage.store, service.lookup, service.register, direct.message.

Each node n publishes signed registrations $\mathrm{Reg}_n \subseteq \{\mathsf{relay}, \mathsf{tcp}, \mathsf{https}\} \cup (\Sigma_{\mathsf{invoke}} \times \Omega)$ carrying its 128-bit process epoch e_n and quota $\mathrm{quota}(n, f, \omega)$, acting as a partial Σ-algebra with generic effect [34], [37]

$$[\![f]\!]_n : \mathrm{Args}_f \times \mathrm{In}_f \to M\,\mathrm{Out}_f.$$

Definition 31 (Kleisli Pipeline Composition (D4a, D4b, Laws L1, L2)). A pipeline $P = (f_1, \bar{a}_1) \ggg \cdots \ggg (f_n, \bar{a}_n)$ is a well-formed morphism of the free category $\mathcal{F}(G_\Sigma)$ iff $1 \leq n \leq n_{\max} = 4$, $\mathrm{Out}_{f_k} = \mathrm{In}_{f_{k+1}}$ for all $k < n$, intermediate symbols satisfy $\mathsf{Intermediate} \in \mathrm{pos}(f_k)$, and world-facing stream symbols ($\mathsf{tcp}, \mathsf{https}$) appear only at $n = 1$. Placed on nodes $h_1, \ldots, h_n$, P denotes the Kleisli composite in $\mathbf{Kl}(M)$:

$$[\![P]\!](v_0) = \big([\![f_1]\!]_{h_1}(\bar{a}_1, -) \ggg \cdots \ggg [\![f_n]\!]_{h_n}(\bar{a}_n, -)\big)(v_0),$$

satisfying Law L1 (Identity): $[\![\mathsf{relay}]\!] = \eta$ and $\mathrm{Loop}(\mathsf{relay} \ggg P) = \mathrm{Loop}(P)$ via normal-form nf reduction, and Law L2 (Associativity): $(f \ggg g) \ggg h = f \ggg (g \ggg h)$.

C. Client-Sealed Loops, Uniform Sphinx Header, and AEZ v5 Carry

Definition 32 (Client-Sealed Loop and Hall-Filter Draw Order (D5, Law L7)). For client $\diamond$, entry guard g, and $n \in [1, 4]$ symbol hops $h_1, \ldots, h_n$, each segment $k \in [0, n]$ inserts $s = 2$ relays $r_{k,1}, r_{k,2}$ (the minimal s such that no node is adjacent to both $\diamond$ and a symbol hop):

$$\begin{aligned}\mathrm{Loop}(P):\ \diamond \to g = r_{0,1} \to r_{0,2} \to h_1 \to r_{1,1} \to r_{1,2} \\ \to \cdots \to h_n \to r_{n,1} \to r_{n,2} = g \to \diamond,\end{aligned}$$

with total hops $H(n, s) = (s+1)n + s \leq \hat{H} = 14$ ($H \in \{5, 8, 11, 14\}$) and $H - 1$ distinct peer DIDs (Law L7: Guard Closure). Route selection draws $h_1, \ldots, h_n$ first, then guard g, then remaining relays $r_{k,j}$ using a bipartite-matching Hall filter $F(\pi, p)$ so that if the first draw succeeds, no subsequent draw can fail, and exit marginals equal exact quality weights $q(d)/q(N_{h_1})$.

Definition 33 (2870-Byte Sphinx Header and 205-Byte Uniform Per-Hop Layer (D6, D6″, Laws L3, L4)). Every

hop $i \in [1, H]$ across all roles (HopRole::Transit | Terminal | GuardReturn) peels an identical 205-byte layer

$$\lambda_i = (f_i, \bar{a}_i, \mathrm{next}_i, e_i, x, \nu_i, \kappa_i, \gamma_{i+1}),$$
$$\ell = 1 + 64 + 20 + 16 + 8 + 16 + 64 + 16 = 205\,\mathrm{B},$$

from a fixed-length Sphinx header [30] $\chi_i = \alpha_i \| \beta_i \| \gamma_i$ with blinded SEC1 group element $|\alpha_i| = 33\,\mathrm{B}$, padded routing block $|\beta_i| = \hat{H}\ell = 14 \times 205 = 2870\,\mathrm{B}$, and MAC $|\gamma_i| = 16\,\mathrm{B}$ ($|\chi_i| = 2919\,\mathrm{B}$), where $x = \lceil t_{\mathrm{build}}/Q \rceil Q + X_0$ ($Q = 30\,\mathrm{s}$, $X_0 = 120\,\mathrm{s}$, $V = 150\,\mathrm{s}$). Each loop selects a cell bucket class $b \in [16\,\mathrm{KiB}, 12\,\mathrm{MiB}]$ with uniform carry slot width $C_b = b - 2919\,\mathrm{B}$ ($C_{16\,\mathrm{KiB}} = 13\,465\,\mathrm{B}$, 82% payload efficiency, Law L3) and deterministic release time $t_i^{\mathrm{dep}} = t_i^{\mathrm{arr}} + L(f_i)$ (Law L4).

Definition 34 (AEZ v5 Wide-Block Body Carry (OnionBodyCipher, D7)). For each segment k, the client draws a 32-byte seed σ_k and derives 48-byte AEZ v5 keys [32], [38] $k_{r_{k,j}}$ and k_{c_k} via HKDF-SHA256. Producer p_k applies one authenticated wide-block encipherment $\mathrm{E}^{\tau}_{k_{c_k}}$ ($\tau = 16$ zero-byte authenticator) followed by $s = 2$ length-preserving wide-block encipherments $\mathrm{E}^{0}_{k_{r_{k,j}}}$ ($\tau = 0$):

$$y_{k,0} = \mathrm{E}^{0}_{k_{r_{k,1}}} \circ \mathrm{E}^{0}_{k_{r_{k,2}}} \circ \mathrm{E}^{16}_{k_{c_k}}\big(\mathrm{pad}(\mathrm{enc}(v_k))\big),$$
$$y_{k,j} = \mathrm{D}^{0}_{k_{r_{k,j}}}(y_{k,j-1}), \qquad b_k = \mathrm{D}^{16}_{k_{c_k}}(y_{k,2}).$$

D. SURB Batching, Cover Loops, Replay Admission, and Security Laws

Law 20 (SURB Batching, Constant-Rate Cover, and Sliced-Bloom Replay (D8, Law L9)). 1) SURB batching: A Single-Use Reply Block for world-facing session ς is the 2979-byte client-sealed suffix $\upsilon = (\mathrm{next}, \chi_\upsilon, \sigma_\upsilon, x_\upsilon)$ [39], [40]. A 16 KiB credit loop batches $k = \lfloor 13\,443/2979 \rfloor = 4$ SURBs in its carry plus its own suffix ($k+1 = 5$ reply credits per forward loop, $Q_{\max} = 256$).

2) Constant-rate cover loops: Active links emit at constant rate $r = \rho B/V$ units/s ($\rho = 0.9$, substituting cover cells when no real cell is queued) and maintain an idle floor $r_{\mathrm{idle}} = 1$ unit/s, sustaining $\geq 1.32\,\mathrm{MB/s}$ single-client download throughput at $b = 16\,\mathrm{KiB}$.

3) SlotRoster-backed sliced-Bloom replay admission (Law L9): Every received cell is charged $u(b) = b/16\,\mathrm{KiB}$ units against the sender DID's ledger $\Lambda_i(d)$ ($|\operatorname{dom}\Lambda_i| \leq 2\mathcal{R}$, per-sender budget $B = 16\,384$ units, global budget $G = 64B$ over $V/Q = 5$ quanta) before SEC1 decoding and ECDH. Valid tags ν_i are recorded in expiry-quantized sliced Bloom filters [41], [42] $R_i[x]$ ($K = 26$ slices of $M = 23\,680$ bits, block false-positive rate $\leq 2^{-26}$, union FP $\leq 2^{-20}$, max memory $\approx 5.3\,\mathrm{MB}$), ensuring every AEZ key is used at most once across restarts and clock rollbacks.

Theorem 4 (Onion Pipeline Security Laws (Laws L5, L5′, L6, L8)). Under the ideal cipher/AEZ, PRG, MAC, HKDF, and DDH assumptions, every loop built by Definitions 32–34 satisfies:

1) Law L5 (Noninterference): $\pi_i(P) = \pi_i(P') \wedge \sigma(P) = \sigma(P') \implies \mathrm{view}_i(P) \approx_c \mathrm{view}_i(P')$: no hop learns another hop's arguments $\bar{a}_j$, invoke backend name ω_j, or segment value $v_k \neq \hat{v}_i$ [43].
2) Law L5′ (Position Hiding): For loops P, P' of arbitrary lengths $H, H' \leq \hat{H} = 14$, a hop's distinguishing advantage between positions i and i' with matching local projection π_i° is bounded tightly by

$$\mathrm{Adv}_i \;\leq\; \hat{H}\epsilon_{\mathrm{PRG}} + \epsilon_{\mathrm{MAC}} + \epsilon_{\alpha} + \frac{|D_i - D'_{i'}|}{Q}.$$

3) Law L6 (Client Path Authority): next_i is sealed in λ_i by $\diamond$ independently of runtime values v_k, admitting the Arrow composition fragment ($\ggg$) while excluding promiscuous monadic bind ($\gg\!=$) [36].
4) Law L8 (Byte Unlinkability and Tagging Resistance): Consecutive edges carry independent uniform ciphertexts ($\chi_i \neq \chi_{i+1}$, $y_{i-1} \neq y_i$), class relabeling fails $\gamma_i = \mathrm{MAC}_{K_i}(b \| \beta_i)$, and any modification $\tilde{y} \neq y_{k,j}$ is rejected by consumer $\mathrm{D}^{16}_{k_{c_k}}$ with probability $\geq 1 - 2^{-128} - \epsilon_{\mathrm{AEZ}}$ [44].

IX. Decentralized Trust and Ranking — DRanking Bridge

A. Sidecar Witness Ordering

Definition 35 (Sidecar witness and partial order).

$$w = (\mathit{domain}, \mathit{root}, \mathit{height}, \mathit{time}, \mathit{finality}),$$
$$\mathrm{Witness}(msg, w) \iff \mathrm{Include}(H(msg), w.\mathit{root}) = \top \wedge \mathrm{Final}(w) = \top$$
$$w_a \preceq w_b \iff \mathit{domain}(w_a) = \mathit{domain}(w_b) \wedge \mathit{time}(w_a) < \mathit{time}$$
$$\chi : \mathit{Domain}_a \times \mathit{Domain}_b \to \mathit{Order}.$$

B. Counterparty-Signed Service Receipts and the DRanking 2.0 Bridge

In an open network where identity creation is free ($\mathsf{cost}(\mathsf{did}) = 0$), unsigned local behavior counters or naive median/trimmed-mean votes over unverified peer reports are vulnerable to Sybil amplification [45]. Rings 1.0 therefore establishes a cryptographic evidence primitive on the wire—the counterparty-signed Service Receipt (ServiceReceipt)—which bridges directly into the companion DRanking 2.0 verifiable distributed ranking and admission protocol [9].

Definition 36 (Rings 1.0 Service Receipt (ServiceReceipt)). A service receipt is a dual-authenticated tuple

$$\rho = \big\langle \mathsf{ver},\, \mathsf{srv},\, a,\, b,\, k,\, n,\, d_{\mathrm{req}},\, d_{\mathrm{rsp}},\, \sigma_a,\, \sigma_b \big\rangle,$$

signed over the canonical domain separator `rings-service-receipt-v1\0`, where:

- $\mathsf{srv} \in \{\mathsf{relay}, \mathsf{store}, \mathsf{probe}\}$ names the metered service kind;
- $a \in R$ is the service provider DID and $b \in R$ ($b \neq a$) is the served beneficiary DID;
- $k \in \mathbb{N}$ is the epoch index and n is a fresh per-epoch nonce;
- $d_{\mathrm{req}}, d_{\mathrm{rsp}}$ are the cryptographic digests of the exact signed request and completion transactions; and

- σ_a, σ_b verify under the distinct account keys of a and b over $\texttt{rings-service-receipt-v1}\backslash 0 \| \mathsf{ver} \| \mathsf{srv} \| a \| b \| k \| n \| d_{\mathrm{req}} \| d_{\mathrm{rsp}}$.

Proposition 5 (DRanking 2.0 Bridge Morphism [9]). Because forging σ_b for an honest beneficiary $b \notin \mathsf{Adv}$ is cryptographically infeasible, manufacturable receipts are restricted to adversarial self-dealing ($b \in \mathsf{Adv}$). In the companion DRanking 2.0 protocol [9], verified 1.0 receipts fold into a cap-budgeted receipt matrix $W_k[b,a] \le w_{\max}$ with normalized stochastic flow $C_k[b,a] = W_k[b,a] / \max(w_{\max}, W_k[b,\cdot]\mathbf{1})$, driving the seed-anchored attenuated PageRank/EigenTrust [46], [47] epoch evolution

$$t_{k+1} = \alpha_k \, C_k^{\mathsf{T}} t_k + (1-\alpha_k)\, s,$$
$$T_k(\mathsf{Adv}) = \sum_{v \in \mathsf{Adv}} t_k(v) \;\le\; \frac{\alpha g}{1-\alpha},$$

where $g = \sup_k \sum_{b \in H, a \in \mathsf{Adv}} C_k[b,a] t_k(b)$ is the honest endorsement leakage. Soulbound rank $\hat{t}_k(v)$ then weights VRF committee sortition and is folded across epochs via Nova-family Incrementally Verifiable Computation (IVC) [48] so browser nodes verify global admission state in $O(1)$ time.

C. Security Predicates and Controls

Attack	Predicate	Rings 1.0 + 2.0 Control
Sybil	many DIDs, one actor	SlotRoster cleanest-first floors + fair pool (Sec. VII); DRanking receipts [9]
Replay	stale valid σ	durable $(\mathcal{W}_s, \mathcal{K}_s)$, link-local $\mathcal{W}_{s,g}$, onion sliced-Bloom $R_i[x]$
MITM	wrong key for DID	DID proof, rings-delegation-v1, rings-data-carrier-v1
DoS	unbounded resource use	5-table slot accounting, credit reclaim Shrink, credit-time $\hat{\tau}$ floor
Tagging	onion byte correlation	Sphinx 2870 B header + AEZ v5 wide-block carry ($\tau = 16$)

Table III
Security predicates and controls across Rings layers

Obligation 2 (Replay rejection).

$$\begin{aligned} \mathrm{accept}(s,\eta,t) &\Rightarrow \eta \notin \mathit{Seen}_s \wedge t < \mathit{exp}(s), \\ \mathit{Seen}'_s &= \mathit{Seen}_s \cup \{\eta\}, \\ \mathsf{emitDelivery} &< \mathit{Seen}'_s. \end{aligned}$$

X. Evaluations and Formal Verification

A. TLA-Style State Relation and Effect Refinement

Definition 37 (State relation and Stateright exploration).

$$C = (S, Q), \qquad \frac{Q = a :: Q_0 \quad I_A(a) = i \quad \tau(S,i) = (S', O, \beta)}{(S,Q) \to (S', Q_0 \cdot \beta)}$$
$$\mathsf{Explore} = \mathsf{TLAStyle}(\to).$$

The exploration relation follows TLA+ style and Stateright finite-state model checking [8], [49] directly over the pure Rust transition functions of rings-core and rings-node.

Invariant 4 (Effect refinement).

$$\mathrm{Impl}_A(s,i) \preceq I_A^\sharp(\tau(s,i)),$$
$$\pi_{state}(\mathrm{Impl}_A) = \pi_{state}(I_A^\sharp \circ \tau).$$

Obligation 3 (Topology coverage).

$$\begin{aligned} \mathcal{L} &= \{\mathit{line}, \mathit{wrap}, \mathit{gap}, \mathit{partition}, \mathit{merge}, \\ &\qquad \mathit{churn}, \mathit{two_clique_bridge}\}, \\ \mathcal{S} &= \{\mathit{loss}, \mathit{dup}, \mathit{reorder}, \mathit{timeout}, \mathit{restart}\}, \\ \mathsf{CheckCase} &= (L, S, P), \qquad L \in \mathcal{L},\ S \in \mathcal{S}. \end{aligned}$$

Definition 38 (Latency decomposition).

$$T_{total} = T_{conn} + h \cdot T_{hop} + T_{crypto} + T_{app} + T_{witness},$$
$$h = O(\log |N|) \quad (F_n \text{ range-proved}).$$

B. Model-Checked State Spaces and Discharged Subsystem Obligations

Unlike legacy analytical estimates copied from idealized simulations, Rings 1.0 discharges its safety, liveness, crash-consistency, and resource-accounting invariants directly against the shipped transition reducers using Stateright BFS model checkers, exhaustive state-space explorers, and deterministic multi-node transport matrices (Table IV).

C. Analytical and Operational Complexity Bounds

Table V summarizes the closed-form analytical and operational bounds established across the Rings 1.0 architecture.

Obligation 4 (Evaluation morphism).

$$\begin{aligned} \Phi_{eval} :& (h, \mathit{timeouts}, \mathit{failures})_{Chord} \\ &\mapsto (T_{total}, \mathit{timeoutMean}, \mathit{failrate})_{Rings}, \end{aligned}$$
$$\Delta\Phi = T_{conn} + T_{crypto} + T_{witness} + T_{app}.$$

XI. Related Work

Distributed hash tables give Rings its closest technical ancestry. IPFS and libp2p use Kademlia-derived routing, where distance is measured by XOR and routing state is organized into buckets [50]–[52]. This is a strong fit for content-addressed lookup, but Rings needs a different law: the owner of a DID or VID is the first live node clockwise from that point, topology repair follows successor-list and multi-predecessor maintenance, and storage redundancy places affine rotations of an entry key around the ring. The paper therefore uses Chord's circular order and range lemma as the base model [4], [5], applies Zave's CorrectChord results for ring safety [6], [10], incorporates Bamboo's timeout-only backoff principle [13], and solves non-transitive WebRTC clique routing [7], [14] without unstructured gossip or virtual fingers [15], [16].

Relay anonymity systems take a different route. Tor, Mixminion, Loopix, and Nym route traffic over selected relay paths to hide communication metadata [31], [40], [53], [54]. Rings integrates Sphinx fixed-length headers [30], Chaum/Mixminion SURBs [39], [40], Loopix-style client

Table IV
Formal verification, Stateright model checking, and discharged obligations across Rings 1.0 subsystems

Subsystem	Model / Explorer Harness	Verified Invariants and Discharged Bounds
Range-Proved Finger Convergence (2.5IV-B)	Depth-9 resource-law explorer; two-phase claim explorer; 200-node deterministic scheduler	Single attempt owner (Idle | AwR | AwA); monotone evidence epochs $e(n,m)$; zero routed lookups on local range $(n, \mathrm{head}(n)]$; cyclic starvation freedom (next_fix_slot); $\leq K+1$ stabilization and $\leq K$ sync connection bounds; $[1, 11]$ s activation jitter (≤ 30 nodes/s bucket).
Non-Transitive WebRTC Chord (2.5IV-C)	6 Stateright/BFS rejoin & multi-clique models; exhaustive delivery explorer	$\mathrm{Referenced}(S_n) \subseteq \mathrm{Admitted}(n)$; articulation retention of $m \leq K$ sub-clique predecessors ($\mathrm{Preds}_K(n)$); sub-clique local finger proof; DialBackoff negative cache ($B_{\max} = 64$); RoutesClockwise; Linked, NeverOvershoots, SingleCrossing, Acyclic ($\leq 2\|V\|$ hops), and $\leq$ 2-hop ($\leq$ 3-hop pre-sync) all-pairs two-clique delivery.
CRDT Removal Stability & Provenance (2.5V)	Factored-carrier crash matrix; HLC provenance & tombstone-cap property suite	Per-writer DataProvenance (rings-data-carrier-v1\0); Theorem 2 (No Resurrection) under $t \geq \tau(r) + H + \sigma$; factored eviction $x \mapsto \mathsf{R}(x)$; crash safety when R flushes before E; $T_{\max} = 1024$ refusal (Storage Tombstone Full) with single-message fit $W \approx 33.83\,\mathrm{MB} \leq 60\,\mathrm{MB}$; lock-order fetch-cache linearizability; 0-tombstone LeaseHeartbeat (rings-lease-heartbeat-v1\0).
Durable Local Stores & Replicated Custody (2.5VI)	Exhaustive crash-point step-plan checker (test_durability); r-member chain repair model	Authority lattice $C \times M$; native .p presence markers ($P \subseteq R$, $\mathsf{Missing}(n)$ detection); macOS F_FULLFSYNC, Windows MoveFileExW(MOVEFILE_WRITE_THROUGH), Linux fsync, browser IndexedDB durability:"strict"; group commit put* undo rollback; link-local $\mathcal{W}_{s,g}$ isolation; atomic $(\mathcal{W}_s, \mathcal{K}_s)$ at-least-once relay custody; age-gated ($A_{\min} = 30\,\mathrm{min}$) r-member chain convergence in $r(L+1)$ passes.
Fair Admission, Slot Accounting & Load Bounds (2.5VII)	Finite-state credit-reclaim model checker; sync-storm operation-count matrix	SlotRoster ($\mathcal{R}(4) = 330$, $\mathcal{R}(3) = 328$) cleanest-first Sybil-rotation defense; Theorem 3 across all 5 shared tables; spent-stream eviction ($H_{\mathrm{spent}} = 600\,\mathrm{s}$); Credit/Shrink/Returned hard frame bound $B_F(s) = \mathcal{R}(s)(W+3) + \mathcal{P}_F$ under frame reordering/duplication; single-charge chunk reassembly with credit-time $\hat{\tau}$ floor; per-category barrier; $O((g+k+r)\log n)$ queue complexity.
Sphinx Onion Kleisli Pipeline (2.5VIII)	2941 AEZ v5 KAT vectors + fuzzing; Hall-filter exact enumeration; builder view-equality suite	Laws L1 (identity), L2 (associativity), L3 (slot width C_b), L4 (exact release), L5 (noninterference), L5′ (position hiding across $H \leq \hat{H} = 14$, $\|\beta\| = 2870\,\mathrm{B}$, $\ell = 205\,\mathrm{B}$), L6 (client path authority), L7 (guard closure), L8 (AEZ v5 wide-block byte unlinkability & 2^{-128} tag rejection), L9 ($2\mathcal{R}$ sender ledgers, $K = 26$ sliced Bloom filter FP $\leq 2^{-20}$, pre-ECDH charge, 128-bit epoch reset); $k = 4$ SURB batching with $\geq 1.32\,\mathrm{MB/s}$ loopback download.

loops [31], and AEZ v5 wide-block enciphering [32], [38], [44], [55] directly into its structured overlay as a typed Kleisli pipeline [33]–[37], [56] whose hops are partial Σ-algebras discovered through the DHT.

On the storage and durability side, Rings combines Shapiro et al.'s state-based CRDTs [17] and Herlihy–Wing linearizability [18] with crash-consistent file system and browser IndexedDB commit laws [19], [22], [28]. Finally, WebRTC and WebAssembly provide portable browser transport and execution targets [57], [58] that interpret the exact same pure state transitions as native nodes.

XII. Conclusion

Rings is an algebraic structured peer-to-peer network: structured because ownership, range-proved finger convergence, non-transitive WebRTC bridge routing, affine CRDT storage replication, and age-gated relay custody are defined over CorrectChord's circular overlay, and algebraic because each layer preserves an explicit carrier, authority lattice, slot-accounting law, or Kleisli morphism rather than hiding semantics inside ad hoc network procedures.

$$\mathsf{Rings} = (R, \mathcal{R}_N, \mathbf{Set}/R, \Omega_N, \mathsf{T}_A^{\mathcal{E}}, \Pi, \mathbb{G}, \mathcal{X}).$$

$$\begin{aligned}
\mathsf{Overlay} &= \mathsf{CorrectChord} + \mathsf{RangeFinger} + \mathsf{BridgeRoute},\\
\mathsf{Storage} &= \mathsf{E}(x) \sqcup \mathsf{R}(x),\ \tau_{\mathrm{HLC}} = (w, c, o),\ T_{\max} = 1024,\\
\mathsf{Durability} &= (c, m) \in C \times M,\ \mathrm{put}^*,\ (\mathcal{W}_s, \mathcal{K}_s),\\
\mathsf{Admission} &= \mathsf{SlotRoster}(\mathcal{R}),\ f_T + \lceil \mathcal{P}_T / n \rceil,\\
\mathsf{Privacy} &= \mathbf{Kl}(M)\ \text{Sphinx/AEZ-v5 loops over}\ \Sigma\ (H \leq 14),\\
\mathsf{Trust} &= \mathsf{ServiceReceipt} \to \mathsf{DRanking\ 2.0}.
\end{aligned}$$

References

[1] N. Kshetri, "Privacy and security issues in cloud computing: The role of institutions and institutional evolution," Telecommunications Policy, vol. 37, no. 4–5, pp. 372–386, 2013.

[2] D. Zissis and D. Lekkas, "Addressing cloud computing security issues," Future Generation Computer Systems, vol. 28, no. 3, pp. 583–592, 2012.

[3] N. Kshetri and S. Murugesan, "Cloud computing and EU data privacy regulations," Computer, vol. 46, no. 3, pp. 86–89, 2013.

[4] I. Stoica, R. Morris, D. Karger, M. F. Kaashoek, and H. Balakrishnan, "Chord: A scalable peer-to-peer lookup service for internet applications," in Proceedings of the 2001 Conference on Applications, Technologies, Architectures, and Protocols for Computer Communications (SIGCOMM '01). ACM, 2001, pp. 149–160.

[5] I. Stoica, R. Morris, D. Liben-Nowell, D. R. Karger, M. F. Kaashoek, F. Dabek, and H. Balakrishnan, "Chord: A scalable peer-to-peer lookup protocol for internet applications," IEEE/ACM Transactions on Networking, vol. 11, no. 1, pp. 17–32, 2003.

Table V
Analytical and operational bounds of Rings 1.0

Metric / Subsystem Quantity	Rings 1.0 Bound
Routed finger lookups per full pass	$\leq R \leq \min(L, \lvert V\rvert)$, $\mathbb{E}[R] = O(\log \lvert V\rvert)$
Local finger range $[0, \lfloor \log_2 \delta_n(h) \rfloor]$	0 routed lookups (stabilization proof)
Report connection fan-out (stab / sync)	$\leq K+1$ / $\leq K$ per claimed report
Non-transitive bridge-delegated route	$\leq 2\lvert V\rvert$ hops worst-case; ≤ 2 hops in 2-clique
CRDT carrier tombstones / wire size	$T_{\max} = 1024$; $W \approx 33.83\,\text{MB} \leq 60\,\text{MB}$
Registry heartbeat tombstones / reads	0 tombstones, 0 reads (Extend lease)
Group commit put* flushes / latency	1 drive barrier per batch; release $\leq 2\phi$
Replicated relay chain repair delay	$D \leq T_d + r\delta$ ($T_d \leq 10\,\text{s}$ silent, 0 s graceful)
Connection slot roster capacity $\mathcal{R}(s)$	$2(F+s+1) = 328$ ($s=3$) or 330 ($s=4$)
Node-wide frame liability $B_F(s)$	$\mathcal{R}(s)(W+3) + \mathcal{P}_F = 6\,488$ frames ($s=3$)
Open chunk reassembly buffer $B_O(3)$	$328 \times 256\,\text{KiB} + 128\,\text{MiB} = 210\,\text{MiB}$
Onion loop hops $H(n,s)$ / header $\lvert\chi\rvert$	$H = (s+1)n + s \leq 14$; $\lvert\chi\rvert = 2919\,\text{B}$ ($\ell = 205\,\text{B}$)
Onion replay Bloom filter FP / memory	$\leq 2^{-20}$ ($K = 26, M = 23\,680$); $\leq 5.3\,\text{MB}$
Onion 16 KiB SURB batch / throughput	$k = 4$ (5 credits/loop); $\geq 1.32\,\text{MB/s}$

System	Metric	Ownership and Privacy Law
Kademlia fam.	XOR	bucket routing; no circular interval owner
Relay mixnets	path	onion circuits; no structured resource owner
Rings 1.0	circular R	CorrectChord + non-transitive bridge, factored CRDTs, durable records, slot bounds, Sphinx/AEZ Kleisli loops

Table VI
Routing, ownership, and privacy comparison [50]–[54]

[6] P. Zave, "Reasoning about identifier spaces: How to make Chord correct," IEEE Transactions on Software Engineering, vol. 43, no. 12, pp. 1144–1156, 2017, https://arxiv.org/abs/1610.01140.

[7] M. J. Freedman, K. Lakshminarayanan, and D. Mazières, "Non-transitive connectivity and DHTs," in Proceedings of the 2nd Conference on Real, Large Distributed Systems (WORLDS '05). USENIX Association, 2005, pp. 55–60.

[8] J. Nadal, "Stateright: Model checking for implementing distributed systems," https://www.stateright.rs/, 2026, accessed: July 4, 2026.

[9] R. J. Kung, "The DRanking protocol: Verifiable distributed ranking and sybil-bounded admission for structured peer-to-peer networks," Rings Network Companion Specification (papers/dranking.pdf), 2026.

[10] P. Zave, "How to make Chord correct (using a stable base)," 2015.

[11] K. R. Davis, B. Peabody, and P. J. Leach, "Universally unique identifiers (UUIDs)," Internet Engineering Task Force, RFC 9562, 2024.

[12] C. J. Fidge, "Timestamps in message-passing systems that preserve the partial ordering," in Proceedings of the 11th Australian Computer Science Conference, 1988, pp. 56–66.

[13] S. Rhea, D. Geels, T. Roscoe, and J. Kubiatowicz, "Handling churn in a DHT," in Proceedings of the 2004 USENIX Annual Technical Conference. USENIX Association, 2004, pp. 127–140.

[14] B. Ford, P. Srisuresh, and D. Kegel, "Peer-to-peer communication across network address translators," in Proceedings of the 2005 USENIX Annual Technical Conference. USENIX Association, 2005, pp. 179–192.

[15] D. A. Bryan, B. B. Lowekamp, and C. Jennings, "SOSIMPLE: A serverless, standards-based, P2P SIP communication system," in Proceedings of the First International Workshop on Advanced Architectures and Algorithms for Internet Delivery and Applications (AAA-IDEA '05). IEEE, 2005, pp. 42–49.

[16] C. Jennings, B. Lowekamp, E. Rescorla, S. Baset, and H. Schulzrinne, "Resource location and discovery (RELOAD) base protocol," Internet Engineering Task Force, RFC 6940, 2014.

[17] M. Shapiro, N. Preguiça, C. Baquero, and M. Zawirski, "Conflict-free replicated data types," in Stabilization, Safety, and Security of Distributed Systems, ser. Lecture Notes in Computer Science, vol. 6976. Springer, 2011, pp. 386–400.

[18] M. P. Herlihy and J. M. Wing, "Linearizability: A correctness condition for concurrent objects," ACM Transactions on Programming Languages and Systems, vol. 12, no. 3, pp. 463–492, 1990.

[19] T. S. Pillai, V. Chidambaram, R. Alagappan, S. Al-Kiswany, A. C. Arpaci-Dusseau, and R. H. Arpaci-Dusseau, "All file systems are not created equal: On the complexity of crafting crash-consistent applications," in 11th USENIX Symposium on Operating Systems Design and Implementation (OSDI '14). USENIX Association, 2014, pp. 433–448.

[20] Apple, "fsync(2) and fcntl(2): F_FULLFSYNC," macOS manual pages, 2026, accessed 2026-10-23.

[21] Microsoft, "MoveFileExW function (winbase.h): MOVEFILE_WRITE_THROUGH," https://learn.microsoft.com/en-us/windows/win32/api/winbase/nf-winbase-movefileexw, 2026, accessed 2026-10-23.

[22] W3C Web Applications Working Group, "Indexed database API 3.0," W3C Editor's Draft, https://w3c.github.io/IndexedDB/, 2026, section "Transaction durability"; accessed 2026-10-23.

[23] Chrome for Developers, "A change to the default durability mode in IndexedDB," https://developer.chrome.com/blog/indexeddb-durability-mode-now-defaults-to-relaxed, 2024, default changes from strict to relaxed in Chrome 121; accessed 2026-10-23.

[24] Mozilla, "IndexedDB parent actors (ActorsParent.cpp)," https://github.com/mozilla-firefox/firefox/blob/main/dom/indexedDB/ActorsParent.cpp, 2026, BeginWriteTransaction and GetDefaultSynchronousMode; accessed 2026-10-23.

[25] WebKit, "IndexedDB SQLite backing store (SQLiteIDBBackingStore.cpp)," https://github.com/WebKit/WebKit/blob/main/Source/WebCore/Modules/indexeddb/server/SQLiteIDBBackingStore.cpp, 2026, commitTransaction; accessed 2026-10-23.

[26] SQLite, "Write-ahead logging, and PRAGMA synchronous," https://www.sqlite.org/wal.html, 2026, accessed 2026-10-23.

[27] MDN Web Docs, "Storage quotas and eviction criteria," https://developer.mozilla.org/en-US/docs/Web/API/Storage_API/Storage_quotas_and_eviction_criteria, 2026, accessed 2026-10-23.

[28] C. Blake and R. Rodrigues, "High availability, scalable storage, dynamic peer networks: Pick two," in Proceedings of the 9th Workshop on Hot Topics in Operating Systems (HotOS IX), 2003, pp. 1–6.

[29] A. Shamir, "How to share a secret," Communications of the ACM, vol. 22, no. 11, pp. 612–613, 1979.

[30] G. Danezis and I. Goldberg, "Sphinx: A compact and provably secure mix format," in Proceedings of the 30th IEEE Symposium on Security and Privacy. IEEE, 2009, pp. 269–282.

[31] A. M. Piotrowska, J. Hayes, T. Elahi, S. Meiser, and G. Danezis, "The Loopix anonymity system," in Proceedings of the 26th USENIX Security Symposium. USENIX Association, 2017, pp. 1199–1216.

[32] V. T. Hoang, T. Krovetz, and P. Rogaway, "Robust authenticated-encryption: AEZ and the problem that it solves," in Advances in Cryptology – EUROCRYPT 2015, ser. Lecture Notes in Computer Science, vol. 9056. Springer, 2015, pp. 15–44.

[33] E. Moggi, "Notions of computation and monads," Information and Computation, vol. 93, no. 1, pp. 55–92, 1991.

[34] G. Plotkin and J. Power, "Algebraic operations and generic effects," Applied Categorical Structures, vol. 11, no. 1, pp. 69–94, 2003.

[35] J. Hughes, "Generalising monads to arrows," Science of Computer Programming, vol. 37, no. 1–3, pp. 67–111, 2000.

[36] S. Lindley, P. Wadler, and J. Yallop, "Idioms are oblivious, arrows are meticulous, monads are promiscuous," Electronic Notes in Theoretical Computer Science, vol. 229, no. 5, pp. 97–117, 2011.

[37] G. Plotkin and M. Pretnar, "Handlers of algebraic effects," in Programming Languages and Systems (ESOP 2009), ser. Lecture Notes in Computer Science, vol. 5502. Springer, 2009, pp. 80–94.

[38] V. T. Hoang, T. Krovetz, and P. Rogaway, "AEZ v5: Authenticated encryption by enciphering," CAESAR competition submission, third round, 2017.

[39] D. L. Chaum, "Untraceable electronic mail, return addresses, and digital pseudonyms," Communications of the ACM, vol. 24, no. 2, pp. 84–90, 1981.

[40] G. Danezis, R. Dingledine, and N. Mathewson, "Mixminion: Design of a type III anonymous remailer protocol," in Proceedings of the 2003 IEEE Symposium on Security and Privacy. IEEE, 2003, pp. 2–15.

[41] B. H. Bloom, "Space/time trade-offs in hash coding with allowable errors," Communications of the ACM, vol. 13, no. 7, pp. 422–426, 1970.

[42] P. S. Almeida, C. Baquero, N. Preguiça, and D. Hutchison, "Scalable Bloom filters," Information Processing Letters, vol. 101, no. 6, pp. 255–261, 2007.

[43] J. A. Goguen and J. Meseguer, "Security policies and security models," in Proceedings of the 1982 IEEE Symposium on Security and Privacy. IEEE, 1982, pp. 11–20.

[44] J. P. Degabriele and M. Stam, "Untagging Tor: A formal treatment of onion encryption," in Advances in Cryptology – EUROCRYPT 2018, ser. Lecture Notes in Computer Science, vol. 10822. Springer, 2018, pp. 259–293.

[45] J. R. Douceur, "The sybil attack," in Peer-to-Peer Systems, First International Workshop (IPTPS 2002), ser. Lecture Notes in Computer Science, vol. 2429. Springer, 2002, pp. 251–260.

[46] L. Page, S. Brin, R. Motwani, and T. Winograd, "The PageRank citation ranking: Bringing order to the web," Stanford InfoLab, Tech. Rep. 1999-66, 1999, http://ilpubs.stanford.edu:8090/422/.

[47] S. D. Kamvar, M. T. Schlosser, and H. Garcia-Molina, "The EigenTrust algorithm for reputation management in P2P networks," in Proceedings of the 12th International Conference on World Wide Web (WWW 2003). ACM, 2003, pp. 640–651.

[48] A. Kothapalli, S. Setty, and I. Tzialla, "Nova: Recursive zero-knowledge arguments from folding schemes," in Advances in Cryptology – CRYPTO 2022, ser. Lecture Notes in Computer Science, vol. 13510. Springer, 2022, pp. 359–388.

[49] L. Lamport, Specifying Systems: The TLA+ Language and Tools for Hardware and Software Engineers. Addison-Wesley, 2002.

[50] P. Maymounkov and D. Mazières, "Kademlia: A peer-to-peer information system based on the XOR metric," in Peer-to-Peer Systems, ser. Lecture Notes in Computer Science, vol. 2429. Springer, 2002, pp. 53–65.

[51] J. Benet, "IPFS - content addressed, versioned, P2P file system," arXiv:1407.3561 [cs.NI], 2014, https://arxiv.org/abs/1407.3561.

[52] Protocol Labs, "libp2p: A modular network stack," 2026, https://libp2p.io/.

[53] R. Dingledine, N. Mathewson, and P. Syverson, "Tor: The Second-Generation onion router," in 13th USENIX Security Symposium (USENIX Security 04). San Diego, CA: USENIX Association, Aug. 2004, https://www.usenix.org/conference/13th-usenix-security-symposium/tor-second-generation-onion-router.

[54] C. Diaz, H. Halpin, and A. Kiayias, "The Nym network: The next generation of privacy infrastructure," Whitepaper, 2021, https://nym.com/nym-whitepaper.pdf.

[55] R. Anderson and E. Biham, "Two practical and provably secure block ciphers: BEAR and LION," in Fast Software Encryption (FSE 1996), ser. Lecture Notes in Computer Science, vol. 1039. Springer, 1996, pp. 113–120.

[56] A. Mokhov, G. Lukyanov, S. Marlow, and J. Dimino, "Selective applicative functors," Proceedings of the ACM on Programming Languages, vol. 3, no. ICFP, pp. 90:1–90:29, 2019.

[57] H. Alvestrand, "Overview: Real-time protocols for browser-based applications," RFC 8825, 2021, https://www.rfc-editor.org/rfc/rfc8825.

[58] WebAssembly Working Group, "WebAssembly core specification," W3C Recommendation, 2019, https://www.w3.org/TR/2019/REC-wasm-core-1-20191205/.